Unidad II. Modelos de Procesos de Software

M.I.T.I. Laura López Núñez
Materia: INGENIERÍA DE SOFTWARE I

Contenido

2. MODELOS DE PROCESOS DE SOFTWARE
   - 2.1. Definiciones de modelo de procesos de software
   - 2.2. Modelos tradicionales
     - 2.2.1. Cascada
     - 2.2.2. Modelo en V
   - 2.3. Modelos incrementales
     - 2.3.1. Modelo incremental
     - 2.3.2. DRA (Desarrollo rápido de aplicaciones)
   - 2.4. Modelos evolutivos
     - 2.4.1. Prototipos
     - 2.4.2. Espiral
     - 2.4.3. Desarrollo concurrente
   - 2.5. Proceso unificado
   - 2.6. Modelos ágiles
     - 2.6.1. Principios de los modelos ágiles
     - 2.6.2. Programación extrema (XP)
     - 2.6.3. Método de desarrollo de sistemas dinámicos (DSDM)
     - 2.6.4. Scrum
   - 2.7. Modelos formales
   - 2.8. Otros modelos



Modelos de Procesos de Software

Representación abstracta de un proceso del software.
Proceso desde perspectiva particular.
Proporciona sólo información parcial no son descripciones definitivas de los procesos del software.
Pueden ser extendidos y adaptados para crear procesos más específicos de ingeniería del software.



Modelos de Procesos de Software

- Modelos tradicionales
  - Formados por un conjunto de fases o actividades en las que no tienen en cuenta la naturaleza evolutiva del software
- Modelos incrementales
  - Combina elementos del modelo en cascada con la filosofía interactiva de construcción de prototipos.
- Modelos evolutivos
  - Son modelos que se adaptan a la evolución que sufren los requisitos del sistema en función del tiempo
- Proceso unificado
  - Un proceso de desarrollo de software es el conjunto de actividades necesarias para transformar los requisitos de un usuario en un sistema software.
- Modelos ágiles
  - Son enfoques para el desarrollo de productos que se ajustan a los valores y principios descritos en el Manifiesto Ágil para el desarrollo de software.
- Modelos formales
  - Un modelo matemático del sistema se transforma formalmente en la implementación



Modelos tradicionales

- Las **metodologías tradicionales** o rígidas en el desarrollo del software, son aquellas que establecen una disciplina de trabajo sobre el proceso de desarrollo del software, con el propósito de alcanzar un software más eficiente.

- Se caracterizan por **definir y establecer total y rígidamente todos y cada uno de los requisitos** al inicio de los proyectos de ingeniería de software. Estas metodologías son poco flexibles y no permiten realizar cambios.



Modelos tradicionales

- El método tradicional funciona aplicando un enfoque lineal donde las etapas del transcurso de desarrollo del software deben complementarse secuencialmente. Es decir, una etapa debe completarse antes de que comience la siguiente, dichas etapas reúnen la **recopilación de requisitos y documentación**.



Modelos tradicionales
Modelo de cascada

- El modelo en cascada, algunas veces llamado el ciclo de vida clásico, sugiere un enfoque sistemático, secuencial hacia el desarrollo del software. Fue el primer modelo de proceso de desarrollo de software que se publicó se derivó de procesos de ingeniería de sistemas más generales (Royce, 1970).
- Basado en la mentalidad de línea de ensamblaje
- Es sencillo y fácil de entender
- El proyecto pasa a través de una serie de fases
  - Al final de cada fase se revisan las tareas de trabajo y productos
    - Para poder pasar a la siguiente fase se tiene que haber conseguido todos los objetivos de la fase anterior
  - No hay apenas comunicación entre las fases

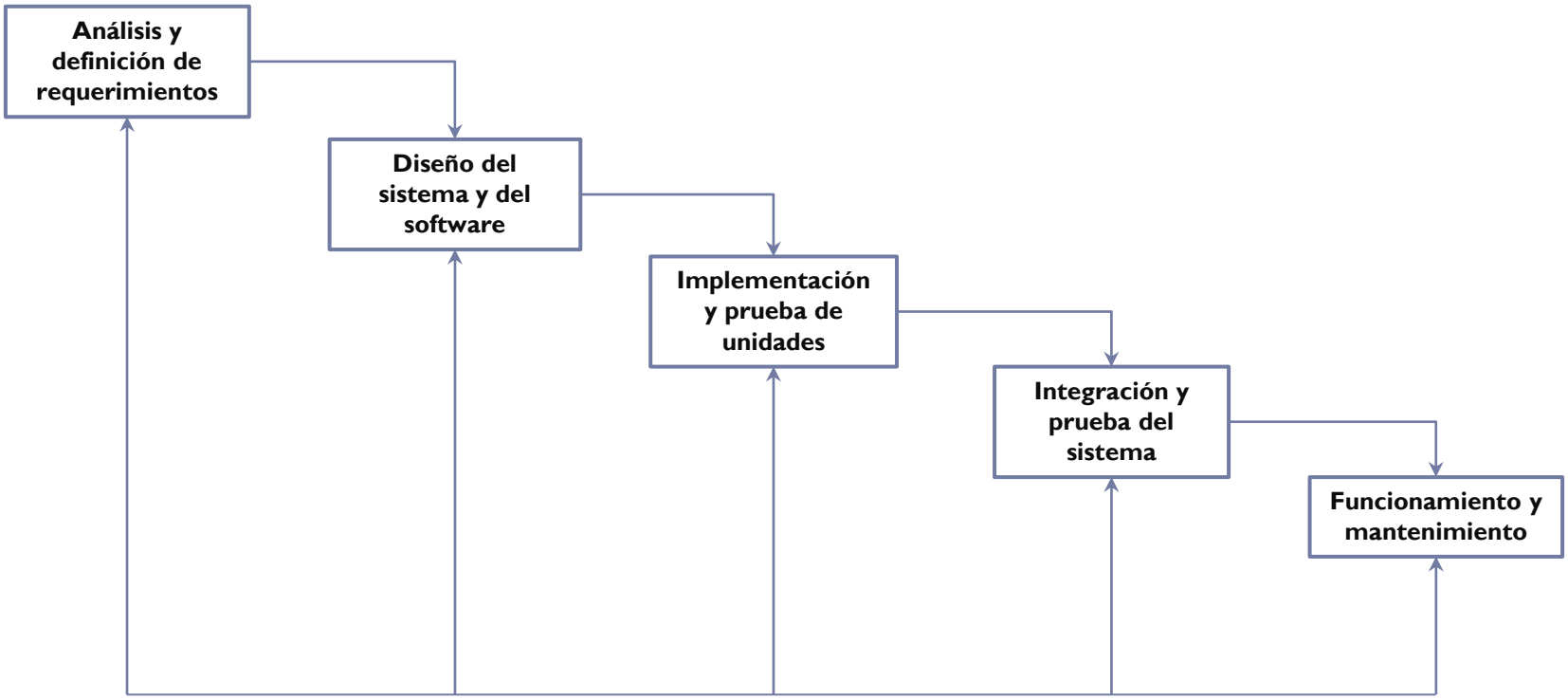


Modelos tradicionales
Modelo de cascada

- Las principales etapas de este modelo se transforman en actividades fundamentales de desarrollo:
  - **Análisis y definición de requerimientos** .- Restricciones y metas del sistema se definen con los usuarios.
  - **Diseño del sistema y del software**.- Divide los requerimientos en sistemas hardware o software El diseño del software identifica y describe las abstracciones fundamentales del sistema software y sus relaciones.
  - **Implementación y prueba de unidades**.- en esta etapa, el diseño del software se lleva a cabo como un conjunto o unidades de programas. La prueba de unidades implica verificar que cada una cumpla su especificación.
  - **Integración y prueba del sistema**.- Unidades individuales se integran y prueban como un sistema completo, después se entrega al cliente.
  - **Funcionamiento y mantenimiento**.- Se instala y se pone en funcionamiento práctico, el mantenimiento corrige errores.



Modelos tradicionales
Modelo de cascada



Modelos tradicionales
Modelo en V

- El ciclo en V, también llamado método en V o modelo en V, se define como un modelo de gestión de proyectos.

- Inspirado en el modelo en cascada (Waterfall model en inglés), representa todo el ciclo de vida de un proyecto y se basa en el mismo principio de gestión secuencial y lineal.

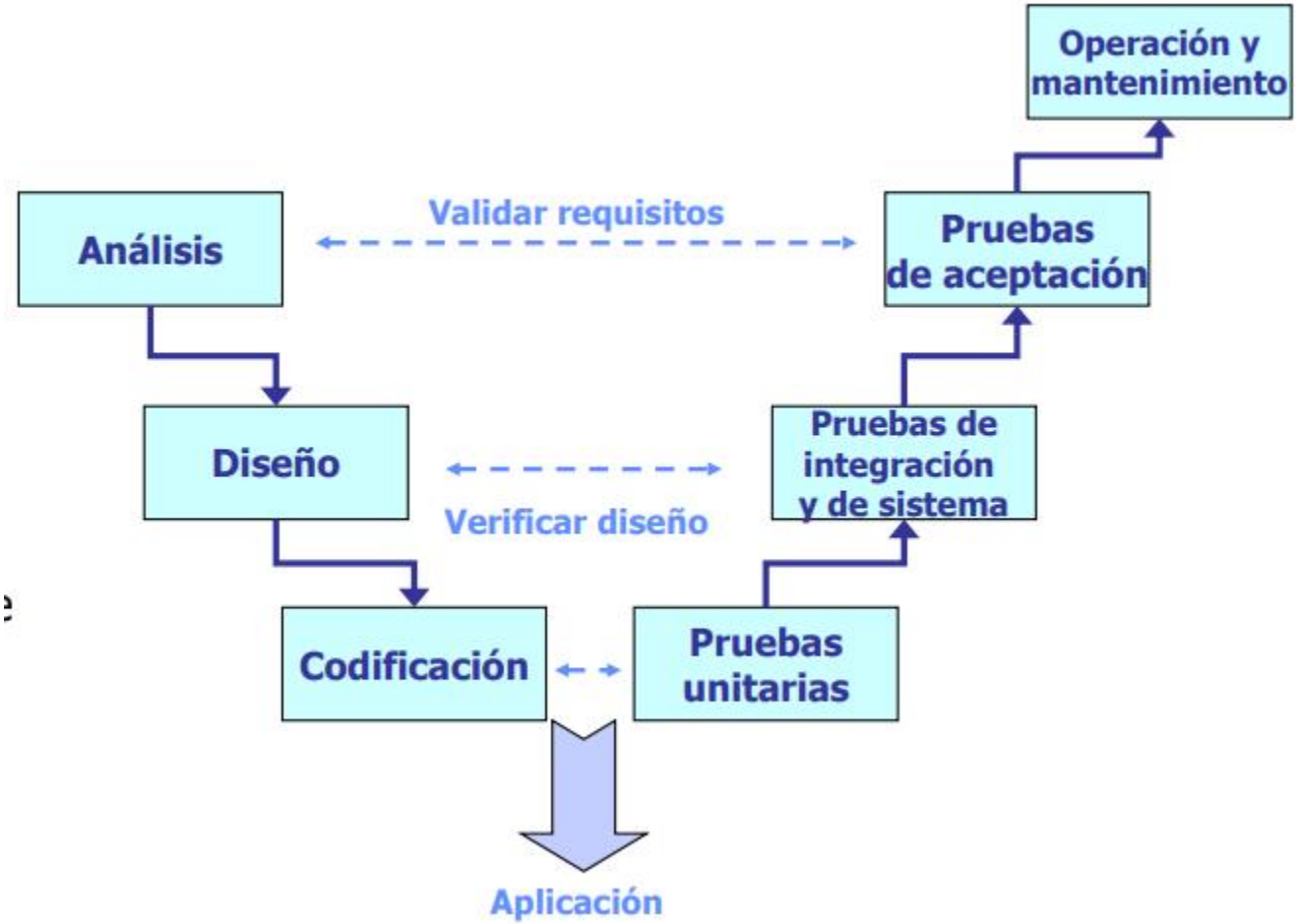


Modelos tradicionales
Modelo en V

- Es una variación del modelo en cascada que demuestra cómo se relacionan las actividades de prueba con las de análisis y desarrollo [GMD, 1992]
- Presenta una implantación ascendente
- Demuestra que el desarrollo de las pruebas se efectúa de manera síncrona con el desarrollo del programa
- Mientras que el modelo clásico centra su atención en los documentos y artefactos producidos, el modelo en V lo hace en la actividad y la exactitud



Modelos tradicionales
Modelo en V



Modelos tradicionales
Modelo en V

- Ventajas
  - Proporciona directrices claras centrándose en una fase a la vez
  - Utiliza un marco sencillo y fácil de entender
  - Establece objetivos específicos para facilitar la delegación de tareas y el seguimiento del progreso
  - Incluye un proceso de revisión de cada fase para garantizar la precisión
  - Promueve el diseño y el desarrollo de alta calidad
  - Incluir una documentación exhaustiva de cada fase



Modelos tradicionales
Modelo en V

- Desventajas
  - Es difícil para el cliente definir todos los requisitos al inicio.
  - No contempla la posibilidad de retornar a etapas anteriores
  - Hasta el final del proyecto no estará disponible una versión del sistema

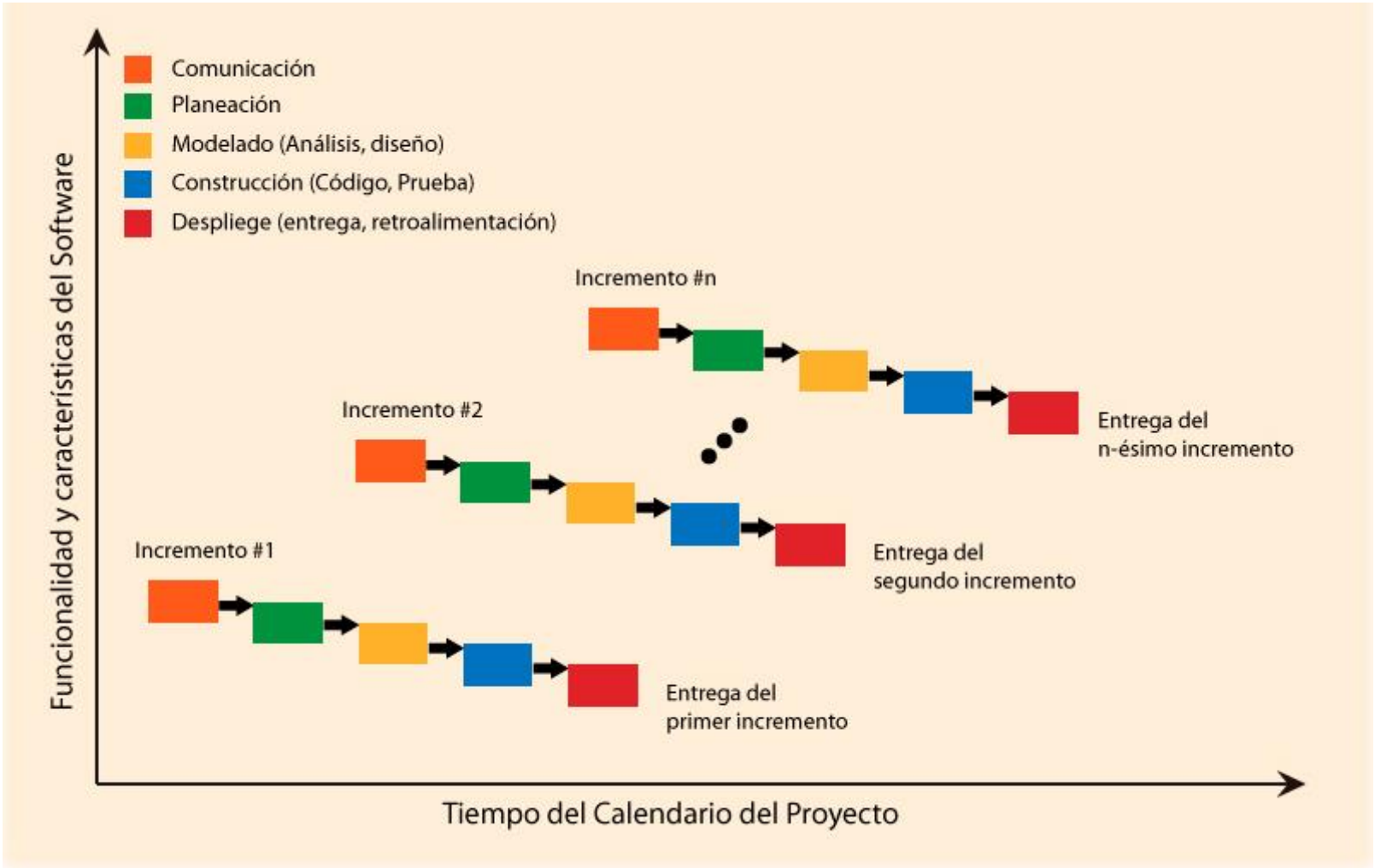


Modelos incrementales

- El modelo incremental combina elementos del modelo en cascada con la filosofía interactiva de construcción de prototipos. Se basa en la filosofía de construir incrementando las funcionalidades del programa. Este modelo aplica secuencias lineales de forma escalonada mientras progresa el tiempo en el calendario. Cada secuencia lineal produce un incremento del software.



Modelos incrementales



Modelos incrementales

- Cuando se utiliza un modelo incremental, el primer incremento es a menudo un producto esencial, sólo con los requisitos básicos. Este modelo se centra en la entrega de un producto operativo con cada incremento. Los primeros incrementos son versiones incompletas del producto final, pero proporcionan al usuario la funcionalidad que precisa y también una plataforma para la evaluación.
- Este modelo es particularmente útil cuando no se cuenta con una dotación de personal suficiente. Los primeros pasos los pueden realizar un grupo reducido de personas y en cada incremento se añadirá personal, de ser necesario



Modelos incrementales

- Durante el proceso se trata de llevar a cabo al proyecto en diferentes partes que al final terminará siendo la solución completa requerida por el cliente, pero éstas partes no se pueden realizar en cualquier orden, sino que dependen de lo que el cliente este necesitando con más urgencia, de los puntos más importantes del proyecto, los requerimientos más básicos, difíciles y con mayor grado de riesgo, ya que estos se deben hacer al comienzo, de manera que se disminuya la dificultad y el riesgo en cada versión.



Modelos incrementales

- El modelo incremental consiste en un desarrollo inicial de la arquitectura completa del sistema, seguido de sucesivos incrementos funcionales. Cada incremento tiene su propio ciclo de vida y se basa en el anterior, sin cambiar su funcionalidad ni sus interfaces.
- Una vez entregado un incremento, no se realizan cambios sobre el mismo, sino únicamente corrección de errores. Dado que la arquitectura completa se desarrolla en la etapa inicial, es necesario conocer los requerimientos completos al comienzo del desarrollo.



Modelos incrementales

- **Características:**
  - Se evitan proyectos largos y se entrega "algo de valor" a los usuarios con cierta frecuencia.
  - El usuario se involucra mas.
  - Difícil de evaluar el costo total.
  - Difícil de aplicar a los sistemas transaccionales que tienden a ser integrados y a operar como un todo.
  - Requiere gestores experimentados.
  - Los errores en los requisitos se detectan tarde.
  - El resultado puede ser positivo.



Modelos incrementales

- **Ventajas**
  - Mediante este modelo se genera software operativo de forma rápida y en etapas tempranas del ciclo de vida del software.
  - Es un modelo más flexible, por lo que se reduce el costo en el cambio de alcance y requisitos.
  - Es más fácil probar y depurar en una iteración más pequeña.
  - Es más fácil gestionar riesgos.
  - Cada iteración es un hito gestionado fácilmente



Modelos incrementales

- **Inconvenientes**
- Para el uso de este modelo se requiere una experiencia importante para definir los incrementos y distribuir en ellos las tareas de forma proporcionada. Entre los inconvenientes que aparecen en el uso de este modelo podemos destacar los siguientes:
  - Cada fase de una iteración es rígida y no se superponen con otras.
  - Pueden surgir problemas referidos a la arquitectura del sistema porque no todos los requisitos se han reunido, ya que se supone que todos ellos se han definido al inicio



Modelos incrementales
DRA (Desarrollo rápido de aplicaciones)

- Es un modelo de proceso de desarrollo de software relativamente corto (dura entre 60 y 90 días).

- Se utiliza la construcción de software basada en componentes, utilizando herramientas de software que permitan de forma ágil y efectiva realizar una aplicación con altos estándares de calidad.



Modelos incrementales
DRA (Desarrollo rápido de aplicaciones)

- El Modelo RAD comprende las siguientes etapas:
  - **Modelado de gestión**: Se genera la información que conduce el proceso de gestión, se identifica a dónde va la información y quién la procesa.
  - **Modelado de datos**: Se definen los almacenes de datos y cómo se relacionan los almacenes entre sí.
  - **Modelado del proceso**: Se utiliza para añadir, modificar, suprimir o recuperar un objeto de datos.

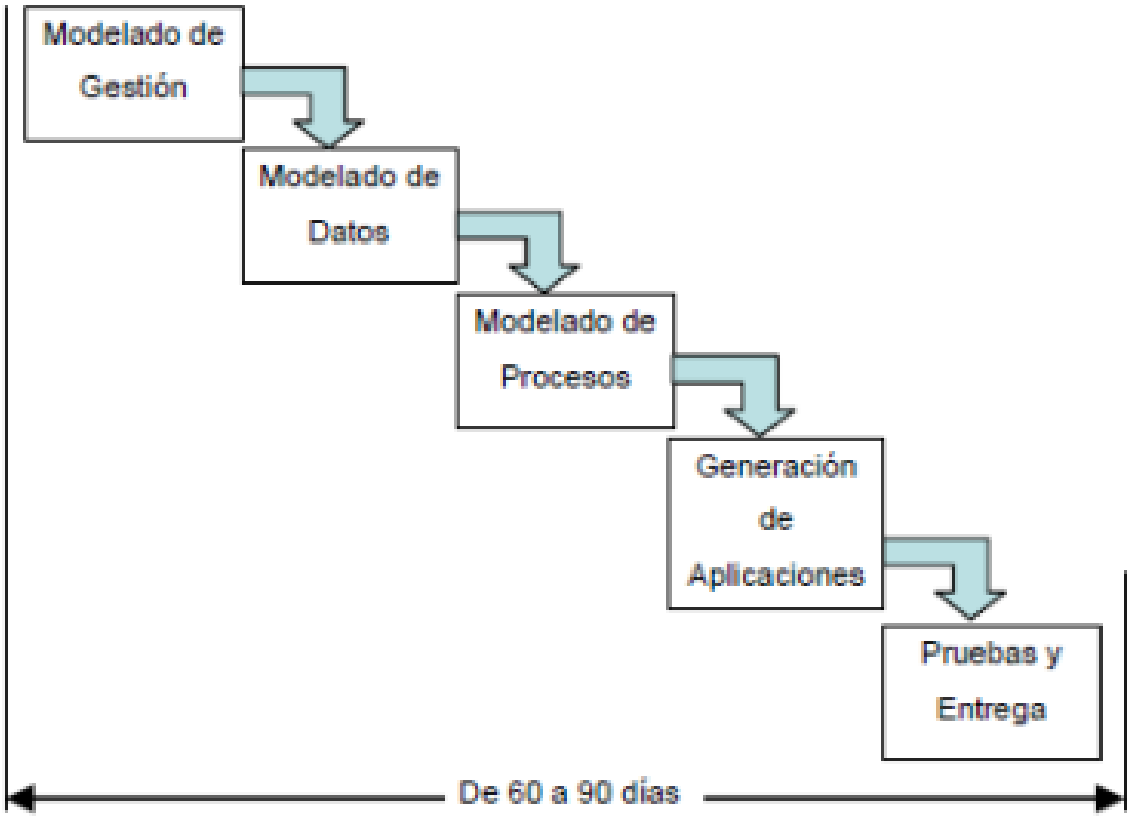


Modelos incrementales
DRA (Desarrollo rápido de aplicaciones)

- El Modelo RAD comprende las siguientes etapas:
  - **Generación de aplicaciones:** Se utiliza una herramienta de cuarta generación que permite crear el software y facilitar la construcción del programa.
  - **Pruebas y entrega:** El proceso de desarrollo finaliza realizando pruebas de calidad del software diseñado con la herramienta RAD, posteriormente se realiza la implementación de la aplicación



Modelos incrementales
DRA (Desarrollo rápido de aplicaciones)



Modelo de Desarrollo Rápido de Aplicaciones.

Modelos incrementales
DRA (Desarrollo rápido de aplicaciones)

- Ventajas
  1. Los usuarios pueden revisar el sistema sistemáticamente.
  2. Los entregables pueden ser fácilmente trasladados a otra plataforma.
  3. El desarrollo se realiza a un nivel de abstracción mayor.
  4. Visibilidad temprana y mayor flexibilidad.
  5. Menor codificación manual y mayor involucramiento de los usuarios.
  6. Posiblemente menos fallas y menor costo
  7. Ciclos de desarrollo más pequeños.



Modelos incrementales
DRA (Desarrollo rápido de aplicaciones)

- Desventajas

1. Para proyectos grandes, aunque por escalas, requiere recursos humanos suficientes como para crear el número correcto de equipos.
2. Requiere clientes y desarrolladores comprometidos en las rápidas actividades. Si no hay compromiso los proyectos fracasaran.
3. Costo de herramientas integradas y equipo necesario.
4. Progreso más difícil de medir.
5. Menos eficiente y menor precisión científica.
6. Dependencia en componentes de terceros: funcionalidad de más o de menos, problemas legales.



Modelos evolutivos

- Desarrollo evolutivo. Este enfoque entrelaza las actividades de especificación, desarrollo y validación.
-
- Un sistema inicial se desarrolla rápidamente a partir de especificaciones abstractas.
- Este se refina basándose en las peticiones del cliente para producir un sistema que satisfaga sus necesidades.



Modelos evolutivos

- El desarrollo evolutivo se basa en la idea de desarrollar una implementación inicial, exponiéndola a los comentarios del usuario y refinándola a través de las diferentes versiones hasta que se desarrolla un sistema adecuado.
- Las actividades de especificación, desarrollo y validación se entrelazan en vez de separarse, con una rápida retroalimentación entre éstas.

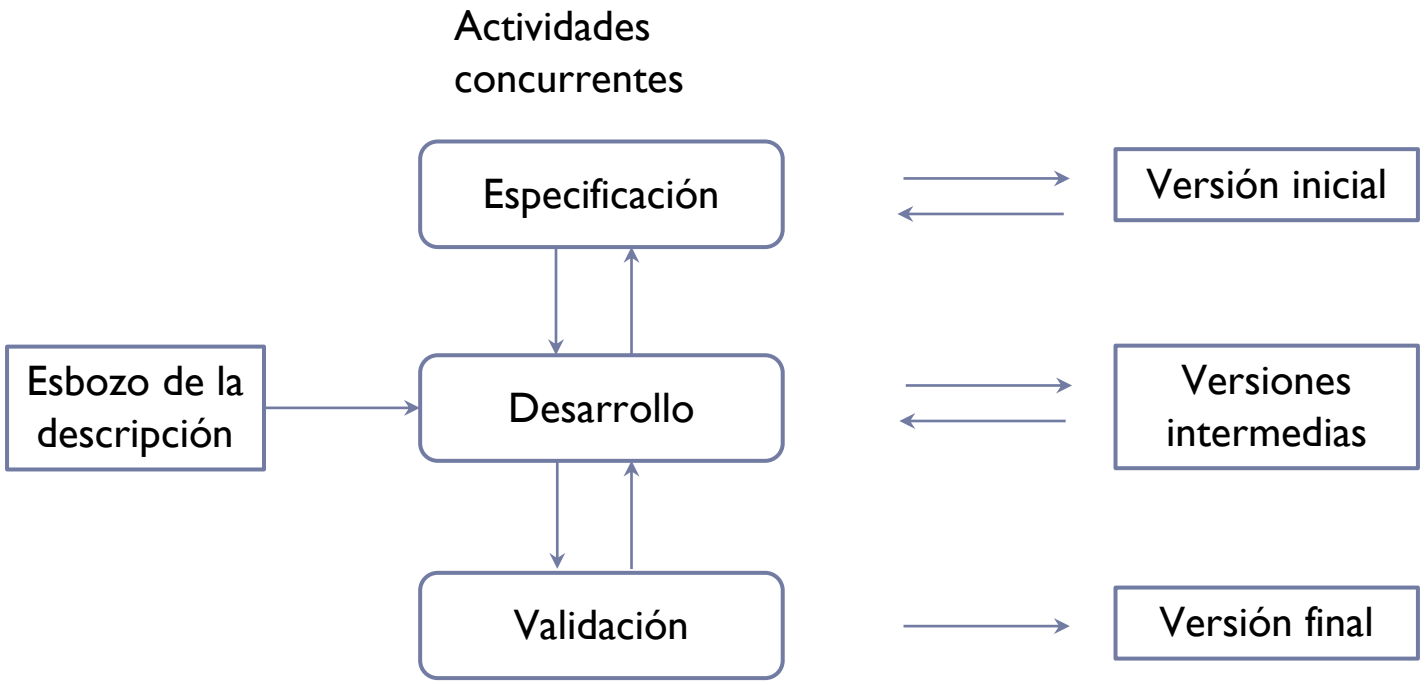


Modelos evolutivos

- Existen dos tipos de desarrollo evolutivo:
  1. **Desarrollo exploratorio**, donde el objetivo del proceso es trabajar con el cliente para explorar sus requerimientos y entregar un sistema final. El desarrollo empieza con las partes del sistema que se comprenden mejor. El sistema evoluciona agregando nuevos atributos propuestos por el cliente.
  2. **Prototipos desechables**, donde el objetivo del proceso de desarrollo evolutivo es comprender los requerimientos del cliente y entonces desarrollar una definición mejorada de los requerimientos para el sistema. El prototipo se centra en experimentar con los requerimientos del cliente que no se comprenden del todo.



Modelos evolutivos



Modelos evolutivos

- En la producción de sistemas, un enfoque evolutivo para el desarrollo de software suele ser más efectivo que el enfoque en cascada, ya que satisface las necesidades inmediatas de los clientes.
- Ventaja de un enfoque evolutivo.
  - La especificación se puede desarrollar de forma creciente. Tan pronto como los usuarios desarrollen un mejor entendimiento de su problema, éste se puede reflejar en el sistema software.



Modelos evolutivos

- **Desventajas importantes**
  1. **El proceso no es visible**. Los administradores tienen que hacer entregas regulares para medir el progreso. Si los sistemas se desarrollan rápidamente, no es rentable producir documentos que reflejen cada versión del sistema.
  2. **A menudo los sistemas tienen una estructura deficiente**. Los cambios continuos tienden a corromper la estructura del software. Incorporar cambios en él se convierte cada vez más en una tarea difícil y costosa.



Modelos evolutivos
Prototipos

- Se generan prototipos en ciclos cortos de desarrollo. El diseño estará orientado a los aspectos visibles para el cliente o usuario final. Esto brinda retroalimentación y ajustes para nuevos prototipos.

- Su objetivo es entender los requisitos del usuario y trabajar para mejorar la calidad de los mismos.



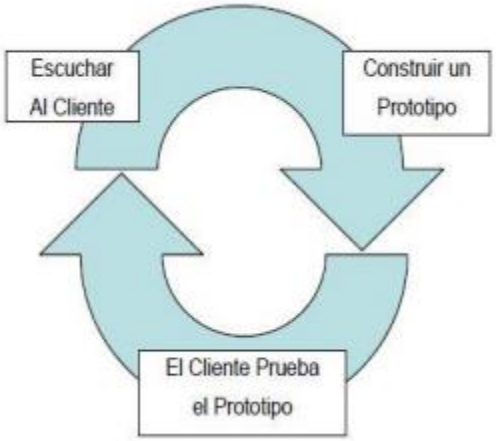
Modelos evolutivos
Prototipos

- Este modelo inicia con la recolección de requerimientos del cliente, con base en estos se define el conjunto de objetivos para el software, se identifican los requisitos conocidos y con base en estos se desarrolla rápidamente un prototipo o maqueta que posteriormente evalúa el cliente utilizándolo y ayudando a refinar de nuevo los requisitos del software a desarrollar; este proceso se seguirá repitiendo hasta que el cliente quede satisfecho con el desarrollo del software (Salazar-Aguirre, y otros, 2011).



Modelos evolutivos
Prototipos

- Modelo Prototipo.



Modelos evolutivos
Prototipos

- Ventajas
  - Genera una buena comunicación con los clientes.
  - Recomendado para proyectos de pequeño y mediano alcance.



Modelos evolutivos
Prototipos

- Desventajas
  - Las planificaciones pudieran sufrir cambios.
  - Dificultades para mantener el sistema en proyectos grandes y en la respuesta a cambios en los requisitos del usuario.
  - Períodos de desarrollo largos.



Modelos evolutivos
Espiral

- Desarrollo en espiral
  - El modelo en espiral del proceso del software fue originalmente propuesto por Boehm (Boehm, 1988).
  - Más que representar el proceso del software como una secuencia de actividades con retrospectiva de una actividad a otra, se representa como una espiral.
  - Cada ciclo en la espiral representa una fase del proceso del software. Así. el ciclo más interno podría referirse a la viabilidad del sistema, el siguiente ciclo a la definición de requerimientos, el siguiente ciclo al diseño del sistema, y así sucesivamente.



Modelos evolutivos
Espiral

- Desarrollo en espiral
- Cada ciclo de la espiral se divide en cuatro sectores:
  1. **Definición de objetivos**. Para esta fase del proyecto se definen los objetivos específicos. Se identifican las restricciones del proceso y el producto, y se traza un plan detallado de gestión. Se identifican los riesgos del proyecto.
  2. **Evaluación y reducción de riesgos**. Se lleva a cabo un análisis detallado para cada uno de los riesgos del proyecto identificados. Se definen los pasos para reducir dichos riesgo. Por ejemplo, si existe el riesgo de tener requerimientos inapropiados, se puede desarrollar un prototipo del sistema.



Modelos evolutivos
Espiral

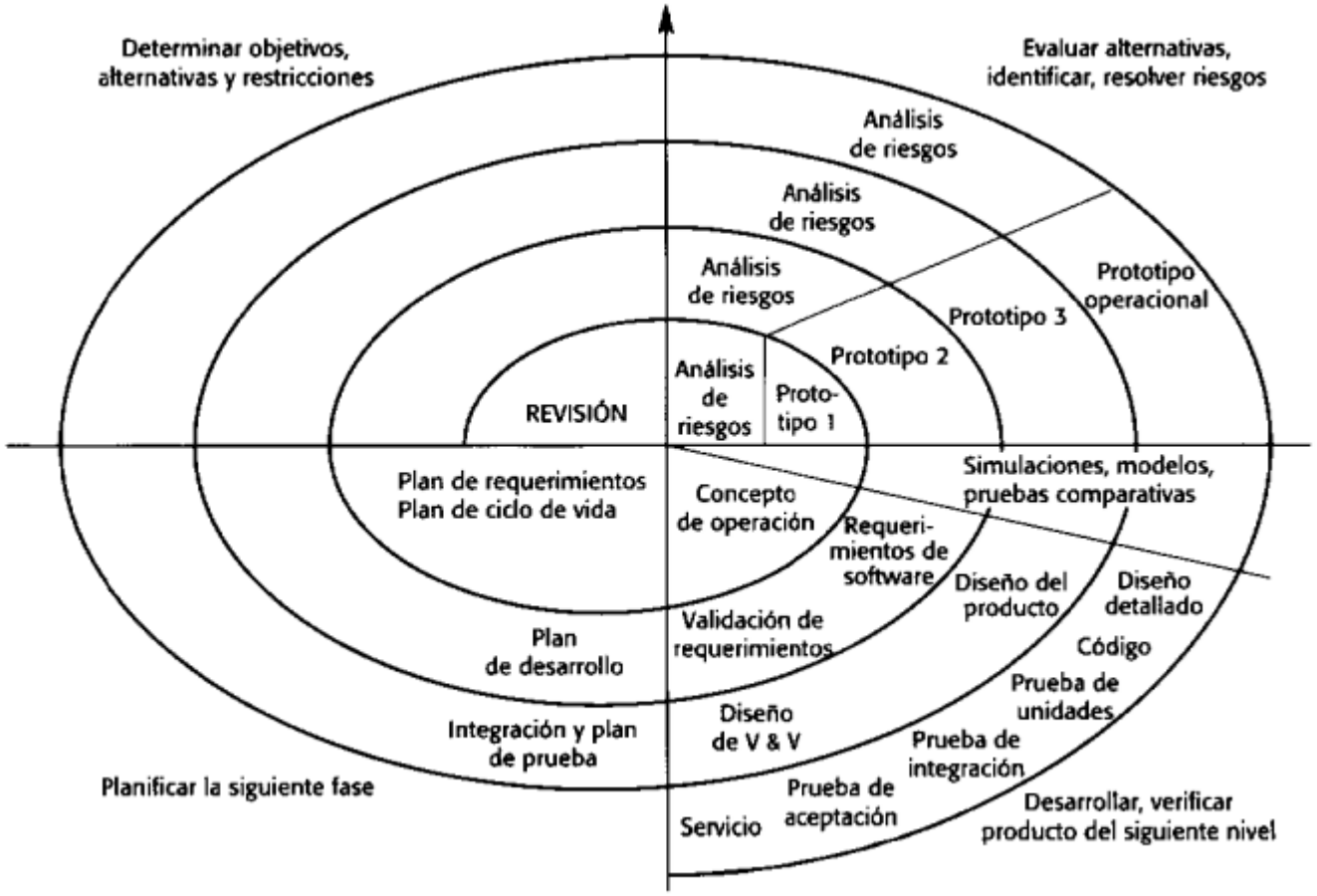
- Desarrollo en espiral
- Cada ciclo de la espiral se divide en cuatro sectores:

3. **Desarrollo y validación**. Después de la evaluación de riesgos, se elige un modelo para el desarrollo del sistema. Por ejemplo, si los riesgos en la interfaz de usuario son dominantes, un modelo de desarrollo apropiado podría ser la construcción de prototipos evolutivos. Si los riesgos de seguridad son la principal consideración, un desarrollo basado en transformaciones formales podría ser el más apropiado, y así sucesivamente. El modelo en cascada puede ser el más apropiado para el desarrollo si el mayor riesgo identificado es la integración de los subsistemas.
4. **Planificación**. El proyecto se revisa y se toma la decisión de si se debe continuar con un ciclo posterior de la espiral. Si se decide continuar, se desarrollan los planes para la siguiente fase del proyecto.



Modelos evolutivos
Espiral

- Desarrollo en espiral



Modelo en espiral de Boehm para el proceso del software (©IEEE, 1988).

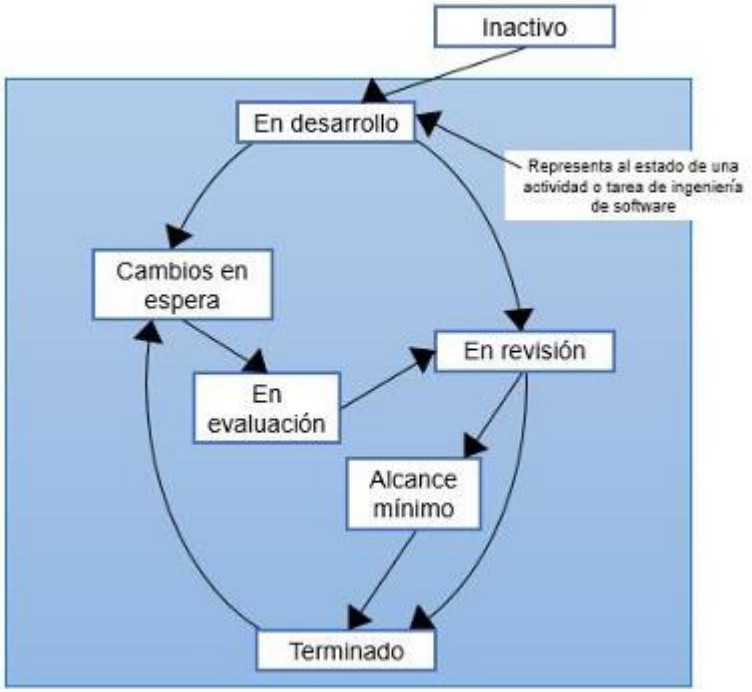
Modelos evolutivos
Espiral

- Desarrollo en espiral
  - Un ciclo de la espiral empieza con la elaboración de objetivos, como el rendimiento y la funcionalidad. Entonces se enumeran formas alternativas de alcanzar estos objetivos y las restricciones impuestas en cada una de ellas.
  - Cada alternativa se evalúa contra cada objetivo y se identifican las fuentes de riesgo del proyecto. El siguiente paso es resolver estos riesgos mediante actividades de recopilación de información como la de detallar más el análisis, la construcción de prototipos y la simulación.
  - Una vez que se han evaluado los riesgos, se lleva a cabo cierto desarrollo. seguido de una actividad de planificación para la siguiente fase del proceso.



Modelos evolutivos
Desarrollo concurrente

- El modelo de desarrollo concurrente, en ocasiones llamado ingeniería concurrente, permite que un equipo de software represente elementos iterativos y concurrentes.



Modelos evolutivos
Desarrollo concurrente

- La actividad -modelado- puede estar en cualquiera de los estados mencionados en un momento dado.
- En forma similar, es posible representar de manera análoga otras actividades, acciones o tareas (por ejemplo, comunicación o construcción). Todas las actividades de ingeniería de software existen de manera concurrente, pero se hallan en diferentes estados.



Modelos evolutivos
Desarrollo concurrente

- La actividad de comunicación termina su primera iteración al principio de un proyecto y existe en el estado de cambios de espera.
- La actividad de modelado, ahora hace una transición al estado en desarrollo, la actividad de modelado pasa del estado en desarrollo al de cambios de espera.



Modelos evolutivos
Desarrollo concurrente

- Ventajas
  - Permite la realización simultánea de todas las tareas de desarrollo hasta la fabricación del prototipo.
  - Funciona utilizando equipos de trabajo multidisciplinarios teniendo un gran énfasis en el manejo de rutas de información más que de jerarquías organizacionales.
  - Es aplicable a todo tipo de desarrollo de software y proporciona una imagen exacta del estado actual de un proyecto.



Modelos evolutivos
Desarrollo concurrente

- Desventajas:
  - Define una red de actividades, donde todas las actividades de la red existen simultáneamente con otras.



Proceso unificado

- **Proceso Unificado de Desarrollo (RUP):** es una metodología de desarrollo de software que está basado en componentes e interfaces bien definidas, y junto con el Lenguaje Unificado de Modelado (UML), constituye la metodología estándar más utilizada para el análisis, implementación y documentación de sistemas orientados a objetos.
- Es un proceso que puede especializarse para una gran variedad de sistemas de software, en diferentes áreas de aplicación, diferentes tipos de organizaciones, diferentes niveles de aptitud y diferentes tamaños de proyecto.



Proceso unificado

- Es un proceso de desarrollo de software
  - Dirigido por casos de uso
  - Centrado en la arquitectura
  - Iterativo e incremental
- Utiliza UML para definir los modelos del sistema software
- El sistema software en construcción está formado por
  - componentes software
  - interconectados a través de interfaces



Proceso unificado

- Es un proceso de desarrollo de software
  - **Dirigido por casos de uso**. Los Casos de Uso son una técnica de captura de requisitos que fuerza a pensar en términos de importancia para el usuario y no sólo en términos de funciones que seria bueno contemplar. Se define un Caso de Uso como un fragmento de funcionalidad del sistema que proporciona al usuario un valor añadido. Los Casos de Uso representan los requisitos funcionales del sistema
  - **Centrado en la arquitectura**. La arquitectura de un sistema es la organización o estructura de sus partes más relevantes, lo que permite tener una visión común entre todos los involucrados así como una perspectiva clara del sistema completo, necesaria para controlar el desarrollo. La arquitectura involucra los aspectos estáticos y dinámicos más significativos del sistema, está relacionada con la toma de decisiones que indican la forma de construcción del sistema. La arquitectura se ve influenciada por la plataforma software, sistema operativo, gestor de bases de datos, protocolos, consideraciones de desarrollo como sistemas heredados. Muchas de estas restricciones constituyen requisitos no funcionales del sistema.
  - **Iterativo e incremental**. en donde el trabajo se divide en partes más pequeñas o mini proyectos, permitiendo generar un equilibrio entre casos de uso y arquitectura. Cada mini proyecto se puede ver como una iteración del cual se obtiene un incremento que produce un crecimiento en el producto.



Proceso unificado

- Como RUP es un proceso, en su modelación define como sus principales elementos:
  - **Trabajadores ("quién"):** Define el comportamiento y responsabilidades (rol) de un individuo, grupo de individuos, sistema automatizado o máquina, que trabajan en conjunto como un equipo. Ellos realizan las actividades y son propietarios de elementos.
  - **Actividades ("cómo"):** Es una tarea que tiene un propósito claro, es realizada por un trabajador y manipula elementos.
  - **Artefactos ("qué"):** Productos tangibles del proyecto que son producidos, modificados y usados por las actividades. Pueden ser modelos, elementos dentro del modelo, código fuente y ejecutables.
  - **Flujo de actividades ("cuándo"):** Secuencia de actividades realizadas por trabajadores y que produce un resultado de valor observable.



Proceso unificado

- **Características Principales de RUP**
  - Unifica los mejores elementos de metodologías anteriores.
  - Preparado para desarrollar grandes y complejos proyectos.
  - Orientado a Objetos.
  - Utiliza el UML como lenguaje de representación visual.
- El RUP reconoce que los modelos de procesos genéricos presentan un sola enfoque del proceso. En contraste, el RUP se describe normalmente desde tres perspectivas:
  - Una perspectiva ***dinámica*** que muestra las fases del modelo sobre el tiempo.
  - Una perspectiva ***estática*** que muestra las actividades del proceso que se representan.
  - Una perspectiva ***práctica*** que sugiere buenas prácticas a utilizar durante el proceso.



Proceso unificado

- El RUP es un modelo en fases que identifica cuatro fases diferentes en el proceso del software. Sin embargo, a diferencia del modelo en cascada donde las fases se equiparan con las actividades del proceso, las fases en el RUP están mucho más relacionadas con asuntos de negocio más que técnicos.
- Fases del RUP
  - Inicio.
  - Elaboración
  - Construcción
  - Transición



Proceso unificado

- Fases del RUP
  - **Inicio.** El objetivo de la fase de inicio es el de establecer un caso de negocio para el sistema. Se deben identificar todas las entidades externas (personas y sistemas) que interactuarán con el sistema y definir estas interacciones. Esta información se utiliza entonces para evaluar la aportación que el sistema hace al negocio. Si esta aportación es de poca importancia, se puede cancelar el proyecto después de esta fase.



Proceso unificado

- Fases del RUP
  - **Elaboración.** Los objetivos de la fase de elaboración son desarrollar una comprensión del dominio del problema, establecer un marco de trabajo arquitectónico para el sistema, desarrollar el plan del proyecto e identificar los riesgos clave del proyecto. Al terminar esta fase, se debe tener un modelo de los requerimientos del sistema (se especifican los casos de uso UML), una descripción arquitectónica y un plan de desarrollo del software.



Proceso unificado

- Fases del RUP
  - **Construcción**. La fase de construcción fundamentalmente comprende el diseño del sistema, la programación y las pruebas. Durante esta fase se desarrollan e integran las partes del sistema. Al terminar esta fase, debe tener un sistema software operativo y la documentación correspondiente lista para entregarla a los usuarios.



Proceso unificado

- Fases del RUP
  - **Transición**. La fase final del RUP se ocupa de mover el sistema desde la comunidad de desarrollo a la comunidad del usuario y hacerlo trabajar en un entorno real. Esto se deja de lado en la mayor parte de los modelos de procesos del software pero es, en realidad, una actividad de alto costo y a veces problemática. Al terminar esta fase, se debe tener un sistema software documentado que funciona correctamente en su entorno operativo.



Proceso unificado



Proceso unificado

- La ventaja de presentar perspectivas dinámicas y estáticas es que las fases del proceso de desarrollo no están asociadas con flujos de trabajo específicos.
- Al menos en principio, todos los flujos de trabajo del RUP pueden estar activos en todas las etapas del proceso. Por supuesto, la mayor parte del esfuerzo se realizará en flujos de trabajo tales como el modelado del negocio y los requerimientos en las primeras fases del proceso y en las pruebas y despliegue en las fases posteriores.



Proceso unificado

- RUP identifica las seis mejores prácticas con las que define una forma efectiva de trabajar para los equipos de desarrollo de software:
  1. El desarrollo iterativo
  2. La administración de requerimientos
  3. La arquitectura basada en componentes
  4. El modelo visual
  5. La verificación continua de la calidad
  6. La administración del cambio



Proceso unificado

1. **Desarrolle el software** de forma iterativa. Planifique incrementos del sistema basado en las prioridades del usuario y desarrollo y entregue las características del sistema de más alta prioridad al inicio del proceso de desarrollo.
2. **Gestione los requerimientos.** Documente explícitamente los requerimientos del cliente y manténgase al tanto de los cambios de estos requerimientos. Analice el impacto de los cambios en el sistema antes de aceptarlos.
3. **Utilice arquitecturas basadas en componentes.** Estructure la arquitectura del sistema en componentes



Proceso unificado

4. **Modele el software visualmente.** Utilice modelos gráficos UML para presentar vistas estáticas y dinámicas del software.
5. **Verifique la calidad del software.** Asegure que el software cumple los estándares de calidad organizacionales.
6. **Controle los cambios del software.** Gestione los cambios del software usando un sistema de gestión de cambios y procedimientos y herramientas de gestión de configuraciones.



Modelos ágiles

- En febrero de 2001, tras una reunión celebrada en Utah-EEUU, nace el término "ágil" aplicado al desarrollo de software. En esta reunión participan un grupo de 17 expertos de la industria del software, incluyendo algunos de los creadores o impulsores de metodologías de software.
- Su objetivo fue esbozar los valores y principios que deberían permitir a los equipos desarrollar software rápidamente y respondiendo a los cambios que puedan surgir a lo largo del proyecto.



Modelos ágiles

- Tras esta reunión se creó The Agile Alliance, una organización, sin ánimo de lucro, dedicada a promover los conceptos relacionados con el desarrollo ágil de software y ayudar a las organizaciones para que adopten dichos conceptos. El punto de partida fue el Manifiesto Ágil, un documento que resume la filosofía "ágil".



Modelos ágiles

- **El Manifiesto Ágil**. Según el Manifiesto se valora:
  - **Al individuo y las interacciones del equipo de desarrollo sobre el proceso y las herramientas**. La gente es el principal factor de éxito de un proyecto software. Es más importante construir un buen equipo que construir el entorno. Muchas veces se comete el error de construir primero el entorno y esperar que el equipo se adapte automáticamente. Es mejor crear el equipo y que éste configure su propio entorno de desarrollo en base a sus necesidades.
  - **Desarrollar software que funciona más que conseguir una buena documentación**. La regla a seguir es "no producir documentos a menos que sean necesarios de forma inmediata para tomar una decisión importante". Estos documentos deben ser cortos y centrarse en lo fundamental.



Modelos ágiles

- **El Manifiesto Ágil**
  - **La colaboración con el cliente más que la negociación de un contrato**. Se propone que exista una interacción constante entre el cliente y el equipo de desarrollo. Esta colaboración entre ambos será la que marque la marcha del proyecto y asegure su éxito.
  - **Responder a los cambios más que seguir estrictamente un plan.** La habilidad de responder a los cambios que puedan surgir a los largo del proyecto (cambios en los requisitos, en la tecnología, en el equipo, etc.) determina también el éxito o fracaso del mismo. Por lo tanto, la planificación no debe ser estricta sino flexible y abierta.



Modelos ágiles

- Los valores anteriores inspiran los doce principios del manifiesto:
  - I. La prioridad es satisfacer al cliente mediante tempranas y continuas entregas de software que le aporte un valor.
  - II. Dar la bienvenida a los cambios. Se capturan los cambios para que el cliente tenga una ventaja competitiva.
  - III. Entregar frecuentemente software que funcione desde un par de semanas a un par de meses, con el menor intervalo de tiempo posible entre entregas.
  - IV. La gente del negocio y los desarrolladores deben trabajar juntos a lo largo del proyecto.
  - V. Construir el proyecto en torno a individuos motivados. Darles el entorno y el apoyo que necesitan y confiar en ellos para conseguir finalizar el trabajo.
  - VI. El diálogo cara a cara es el método más eficiente y efectivo para comunicar información dentro de un equipo de desarrollo.



Modelos ágiles

- Los valores anteriores inspiran los doce principios del manifiesto:

VII. El software que funciona es la medida principal de progreso.

VIII. Los procesos ágiles promueven un desarrollo sostenible. Los promotores, desarrolladores y usuarios deberían ser capaces de mantener una paz constante.

IX. La atención continua a la calidad técnica y al buen diseño mejora la agilidad.

X. La simplicidad es esencial.

XI. Las mejores arquitecturas, requisitos y diseños surgen de los equipos organizados por sí mismos.

XII. En intervalos regulares, el equipo reflexiona respecto a cómo llegar a ser más efectivo, y según esto ajusta su comportamiento.



Modelos ágiles

- Características de un desarrollo ágil
  - Proceso iterativo e incremental
  - Mitigación del riesgo mediante iteraciones fijas
  - Mejora continua
  - Calidad desde el primer día
  - Priorización de requerimientos de acuerdo a su valor
  - Equipos dedicados y auto-gestionados
  - Colaboración continua con el cliente
  - Incorporar al cambio
  - Prácticas de desarrollo modernas



Modelos ágiles

- Algunos métodos ágiles de desarrollo de software:
  - Desarrollo Adaptativo de Software /Adaptive Software Development (ASD).
  - Proceso Unificado Ágil /Agile Unified Process (AUP).
  - Crystal_Clear.
  - Proceso Unificado esencial/Essential Unified Process (EssUP).
  - Feature Driven Development (FDD).
  - Lean Software Development (LSD).
  - Kanban.
  - Open Unified Process (OpenUP).
  - Programación Extrema (XP).
  - Método de desarrollo de sistemas dinámicos (DSDM).
  - Scrum.
  - G300.



Modelos ágiles
Programación Extrema

PROGRAMACIÓN EXTREMA (EXTREME PROGRAMMING, XP)

- XP es una metodología ágil centrada en potenciar las relaciones interpersonales como clave para el éxito en desarrollo de software, promoviendo el trabajo en equipo, preocupándose por el aprendizaje de los desarrolladores, y propiciando un buen clima de trabajo.
- XP se basa en realimentación continua entre el cliente y el equipo de desarrollo, comunicación fluida entre todos los participantes, simplicidad en las soluciones implementadas y coraje para enfrentar los cambios. XP se define como especialmente adecuada para proyectos con requisitos imprecisos y muy cambiantes, y donde existe un alto riesgo técnico.



Modelos ágiles
Programación Extrema

- Los principios y prácticas son de sentido común pero llevadas al extremo, de ahí proviene su nombre. Kent Beck, el padre de XP, describe la filosofía de XP, sin cubrir los detalles técnicos y de implantación de las prácticas. Posteriormente, otras publicaciones de experiencias se han encargado de dicha tarea.



Modelos ágiles
Programación Extrema

- Las Historias de Usuario. Son la técnica utilizada para especificar los requisitos del software. Se trata de tarjetas de papel en las cuales el cliente describe brevemente las características que el sistema debe poseer, sean requisitos funcionales o no funcionales.
- un ejemplo del contenido de la ficha: fecha, tipo de actividad (nueva, corrección, mejora), prueba funcional, número de historia, prioridad técnica y del cliente, referencia a otra historia previa, riesgo, estimación técnica, descripción, notas y una lista de seguimiento con la fecha, estado cosas por terminar y comentarios.
- A efectos de planificación, las historias pueden ser de una a tres semanas de tiempo de programación. Las historias de usuario son descompuestas en tareas de programación (task card) y asignadas a los programadores para ser implementadas durante una iteración.



Modelos ágiles
Programación Extrema

- Roles XP
  - **Programador**. El programador escribe las pruebas unitarias y produce el código del sistema.
  - **Cliente**. Escribe las historias de usuario y las pruebas funcionales para validar su implementación. Además, asigna la prioridad a las historias de usuario y decide cuáles se implementan en cada iteración centrándose en aportar mayor valor al negocio.
  - **Encargado de pruebas (Tester)**. Ayuda al cliente a escribir las pruebas funcionales. Ejecuta las pruebas regularmente, difunde los resultados en el equipo y es responsable de las herramientas de soporte para pruebas.
  - **Encargado de seguimiento (Tracker)**. Proporciona realimentación al equipo. Verifica el grado de acierto entre las estimaciones realizadas y el tiempo real dedicado, para mejorar futuras estimaciones. Realiza el seguimiento del progreso de cada iteración.



Modelos ágiles
Programación Extrema

- Roles XP
  - **Entrenador (Coach)**. Es responsable del proceso global. Debe proveer guías al equipo de forma que se apliquen las prácticas XP y se siga el proceso correctamente.
  - **Consultor**. Es un miembro externo del equipo con un conocimiento específico en algún tema necesario para el proyecto, en el que puedan surgir problemas.
  - **Gestor (Big boss).** Es el vínculo entre clientes y programadores, ayuda a que el equipo trabaje efectivamente creando las condiciones adecuadas. Su labor esencial es de coordinación.



Modelos ágiles
Programación Extrema

- Proceso XP
  1. El cliente define el valor de negocio a implementar.
  2. El programador estima el esfuerzo necesario para su implementación.
  3. El cliente selecciona qué construir, de acuerdo con sus prioridades y las restricciones de tiempo.
  4. El programador construye ese valor de negocio.
  5. Vuelve al paso 1.
- En todas las iteraciones de este ciclo tanto el cliente como el programador aprenden. No se debe presionar al programador a realizar más trabajo que el estimado, ya que se perderá calidad en el software o no se cumplirán los plazos. De la misma forma el cliente tiene la obligación de manejar el ámbito de entrega del producto, para asegurarse que el sistema tenga el mayor valor de negocio posible con cada iteración.



Modelos ágiles
Programación Extrema

- Proceso XP
  - El ciclo de vida ideal de XP consiste de seis fases: Exploración, Planificación de la Entrega (*Release*), Iteraciones, Producción, Mantenimiento y Muerte del Proyecto.
  - **Fase I: Exploración**
  - En esta fase, los clientes plantean a grandes rasgos las historias de usuario que son de interés para la primera entrega del producto. Al mismo tiempo el equipo de desarrollo se familiariza con las herramientas, tecnologías y prácticas que se utilizarán en el proyecto. Se prueba la tecnología y se exploran las posibilidades de la arquitectura del sistema construyendo un prototipo. La fase de exploración toma de pocas semanas a pocos meses, dependiendo del tamaño y familiaridad que tengan los programadores con la tecnología.



Modelos ágiles
Programación Extrema

- Proceso XP
  - **Fase II: Planificación de la Entrega**
  - En esta fase el cliente establece la prioridad de cada historia de usuario, y correspondientemente, los programadores realizan una estimación del esfuerzo necesario de cada una de ellas. Se toman acuerdos sobre el contenido de la primera entrega y se determina un cronograma en conjunto con el cliente. Una entrega debería obtenerse en no más de tres meses. Esta fase dura unos pocos días.
  - Las estimaciones de esfuerzo asociado a la implementación de las historias la establecen los programadores utilizando como medida el punto. Un punto, equivale a una semana ideal de programación. Las historias generalmente valen de 1 a 3 puntos. Por otra parte, el equipo de desarrollo mantiene un registro de la "velocidad" de desarrollo, establecida en puntos por iteración, basándose principalmente en la suma de puntos correspondientes a las historias de usuario que fueron terminadas en la última iteración.
  - La planificación se puede realizar basándose en el tiempo o el alcance. La velocidad del proyecto es utilizada para establecer cuántas historias se pueden implementar antes de una fecha determinada o cuánto tiempo tomará implementar un conjunto de historias. Al planificar por tiempo, se multiplica el número de iteraciones por la velocidad del proyecto, determinándose cuántos puntos se pueden completar. Al planificar según alcance del sistema, se divide la suma de puntos de las historias de usuario seleccionadas entre la velocidad del proyecto, obteniendo el número de iteraciones necesarias para su implementación.



Modelos ágiles
Programación Extrema

- Proceso XP
  - **Fase III: Iteraciones**
  - Esta fase incluye varias iteraciones sobre el sistema antes de ser entregado. El Plan de Entrega está compuesto por iteraciones de no más de tres semanas. En la primera iteración se puede intentar establecer una arquitectura del sistema que pueda ser utilizada durante el resto del proyecto. Esto se logra escogiendo las historias que fuercen la creación de esta arquitectura, sin embargo, esto no siempre es posible ya que es el cliente quien decide qué historias se implementarán en cada iteración (para maximizar el valor de negocio). Al final de la última iteración el sistema estará listo para entrar en producción.
  - Los elementos que deben tomarse en cuenta durante la elaboración del Plan de la Iteración son: historias de usuario no abordadas, velocidad del proyecto, pruebas de aceptación no superadas en la iteración anterior y tareas no terminadas en la iteración anterior. Todo el trabajo de la iteración es expresado en tareas de programación, cada una de ellas es asignada a un programador como responsable, pero llevadas a cabo por parejas de programadores.



Modelos ágiles
Programación Extrema

- Proceso XP
  - **Fase IV: Producción**
  - La fase de producción requiere de pruebas adicionales y revisiones de rendimiento antes de que el sistema sea trasladado al entorno del cliente. Al mismo tiempo, se deben tomar decisiones sobre la inclusión de nuevas características a la versión actual, debido a cambios durante esta fase.
  - Es posible que se rebaje el tiempo que toma cada iteración, de tres a una semana. Las ideas que han sido propuestas y las sugerencias son documentadas para su posterior implementación (por ejemplo, durante la fase de mantenimiento).
  - **Fase V: Mantenimiento**
  - Mientras la primera versión se encuentra en producción, el proyecto XP debe mantener el sistema en funcionamiento al mismo tiempo que desarrolla nuevas iteraciones. Para realizar esto se requiere de tareas de soporte para el cliente. De esta forma, la velocidad de desarrollo puede bajar después de la puesta del sistema en producción. La fase de mantenimiento puede requerir nuevo personal dentro del equipo y cambios en su estructura.



Modelos ágiles
Programación Extrema

- Proceso XP
  - **Fase VI: Muerte del Proyecto**
  - Es cuando el cliente no tiene más historias para ser incluidas en el sistema. Esto requiere que se satisfagan las necesidades del cliente en otros aspectos como rendimiento y confiabilidad del sistema. Se genera la documentación final del sistema y no se realizan más cambios en la arquitectura. La muerte del proyecto también ocurre cuando el sistema no genera los beneficios esperados por el cliente o cuando no hay presupuesto para mantenerlo.



Modelos ágiles
Programación Extrema

- Prácticas XP
- La principal suposición que se realiza en XP es la posibilidad de disminuir la mítica curva exponencial del costo del cambio a lo largo del proyecto, lo suficiente para que el diseño evolutivo funcione. Esto se consigue gracias a las tecnologías disponibles para ayudar en el desarrollo de software y a la aplicación disciplinada de las siguientes prácticas.
  - **El juego de la planificación**. Hay una comunicación frecuente el cliente y los programadores. El equipo técnico realiza una estimación del esfuerzo requerido para la implementación de las historias de usuario y los clientes deciden sobre el ámbito y tiempo de las entregas y de cada iteración.
  - **Entregas pequeñas** . Producir rápidamente versiones del sistema que sean operativas, aunque no cuenten con toda la funcionalidad del sistema. Esta versión ya constituye un resultado de valor para el negocio. Una entrega no debería tardar más 3 meses.



Modelos ágiles
Programación Extrema

- Prácticas XP
  - **Metáfora**. El sistema es definido mediante una metáfora o un conjunto de metáforas compartidas por el cliente y el equipo de desarrollo. Una metáfora es una historia compartida que describe cómo debería funcionar el sistema
  - **Diseño simple**. Se debe diseñar la solución más simple que pueda funcionar y ser implementada en un momento determinado del proyecto.
  - **Pruebas**. La producción de código está dirigida por las pruebas unitarias. Éstas son establecidas por el cliente antes de escribirse el código y son ejecutadas constantemente ante cada modificación del sistema.
  - **Refactorización (Refactoring)**. Es una actividad constante de reestructuración del código con el objetivo de remover duplicación de código, mejorar su legibilidad, simplificarlo y hacerlo más flexible para facilitar los posteriores cambios. Se mejora la estructura interna del código sin alterar su comportamiento externo.



Modelos ágiles
Programación Extrema

- Prácticas XP
  - **Programación en parejas** . Toda la producción de código debe realizarse con trabajo en parejas de programadores. Esto conlleva ventajas implícitas (menor tasa de errores, mejor diseño, mayor satisfacción de los programadores, ...).
  - **Propiedad colectiva del código**. Cualquier programador puede cambiar cualquier parte del código en cualquier momento.
  - **Integración continua**. Cada pieza de código es integrada en el sistema una vez que esté lista. Así, el sistema puede llegar a ser integrado y construido varias veces en un mismo día .
  - **40 horas por semana**. Se debe trabaja r un máximo de 40 horas por semana. No se trabajan horas extras en dos semanas seguidas. Si esto ocurre, probablemente está ocurriendo un problema que debe corregirse. El trabajo extra desmotiva al equipo.
  - **Cliente in-situ**. El cliente tiene que estar presente y disponible todo el tiempo para el equipo. Éste es uno de los principales factores de éxito del proyecto XP. El cliente conduce constantemente el trabajo hacia lo que aportará mayor valor de negocio y los programadores pueden resolver de manera inmediata cualquier duda asociada. La comunicación oral es más efectiva que la escrita.



Modelos ágiles
Programación Extrema

- Prácticas XP
  - **Estándares de programación**. XP enfatiza que la comunicación de los programadores es a través del código, con lo cual es indispensable que se sigan ciertos estándares de programación para mantener el código legible .
- El mayor beneficio de las prácticas se consigue con su aplicación conjunta y equilibrada puesto que se apoyan unas en otras. La mayoría de las prácticas propuestas por XP no son novedosas sino que en alguna forma ya habían sido propuestas en ingeniería del software e incluso demostrado su valor en la práctica. El mérito de XP es integrarlas de una forma efectiva y complementarlas con otras ideas desde la perspectiva del negocio, los valores humanos y el trabajo en equipo.



Modelos ágiles
Método de Desarrollo de Sistemas Dinámicos (DSDM)

- **Dynamic Systems Development Method** (DSDM). Define el marco para desarrollar un proceso de producción de software. Nace en 1994 con el objetivo de crear una metodología RAD unificada. Sus principales características son: es un proceso iterativo e incremental y el equipo de desarrollo y el usuario trabajan juntos. Propone cinco fases: *estudio viabilidad, estudio del negocio, modelado funcional, diseño y construcción, y finalmente implementación.* Las tres últimas son iterativas, además de existir realimentación a todas las fases.



Modelos ágiles

Método de Desarrollo de Sistemas Dinámicos (DSDM)

- DSDM se basa en ocho principios clave. Cada principio respalda la filosofía de DSDM de que "el mejor valor comercial surge cuando los proyectos se alinean con objetivos comerciales claros, se entregan con frecuencia e involucran la colaboración de personas motivadas y empoderadas".



Modelos ágiles
Método de Desarrollo de Sistemas Dinámicos (DSDM)

- **Los ocho principios clave de DSDM**
  - ***Concéntrese en el modelo de negocio***
    - Cada equipo de proyecto debe trabajar de acuerdo con el caso de negocio y no tratar el proyecto como un fin en sí mismo.
  - ***Entregar a tiempo***
    - El timeboxing debe incorporarse en todo momento, incluso en proyectos sin un cronograma fijo o una fecha de finalización.
    - Los plazos autoimpuestos ayudan a un equipo a crear un cronograma de entrega predecible que eleva la moral a través de victorias pequeñas y frecuentes.
  - ***Colabore***
    - La colaboración entre los miembros del equipo y las partes interesadas clave es clave para crear culturas dinámicas y efectivas.



Modelos ágiles
Método de Desarrollo de Sistemas Dinámicos (DSDM)

- **Los ocho principios clave de DSDM**
  - ***Nunca comprometa la calidad***
    - Los estándares de calidad deben definirse y acordarse antes del comienzo del proyecto. Luego, deben mantenerse a través de pruebas, documentación y revisión continuas.
  - ***Construir gradualmente a partir de cimientos firmes***
    - Los equipos de proyecto deben construir una sólida comprensión de los requisitos del proyecto y la solución propuesta. Después de cada incremento del proyecto, o Timebox, es entregado, las prioridades del proyecto y la viabilidad son reevaluadas.



Modelos ágiles
Método de Desarrollo de Sistemas Dinámicos (DSDM)

- **Los ocho principios clave de DSDM**
  - ***Desarrollar iterativamente***
    - DSDM fomenta el desarrollo iterativo basado en los comentarios de los clientes, usuarios y partes interesadas.
    - Los objetivos de desarrollo iterativo deben establecerse con respecto al entorno del proyecto.
  - ***Comunicarse de manera continua y clara.***
    - Siempre que sea posible, DSDM sugiere reuniones presenciales o talleres diarios. Los roles y responsabilidades deben estar claramente definidos.
    - La documentación debe ser ligera y producida de manera oportuna.
    - Esto promueve proyectos altamente transparentes donde las partes interesadas sienten que sus necesidades particulares son escuchadas, comprendidas y atendidas.



Modelos ágiles
Método de Desarrollo de Sistemas Dinámicos (DSDM)

- **Los ocho principios clave de DSDM**
  - ***Demostrar control***
    - Los gerentes de proyecto y los líderes de equipo deben poder demostrar y comunicar el control del proyecto.
    - En otras palabras, el proyecto está totalmente alineado con los objetivos de la empresa.
    - El control también significa que los gerentes de proyecto rastrean métricas importantes periódicamente para mantener la viabilidad del proyecto.



Modelos ágiles
Método de Desarrollo de Sistemas Dinámicos (DSDM)

- Enfoque y fases DSDM
  - Hay seis fases principales en el marco del DSDM:
    1. **Pre-proyecto:** Durante esta fase la organización se asegura de que el proyecto esté alineado con las metas estratégicas y tenga un objetivo claramente definido.
    2. **Viabilidad:** Durante esta fase la organización se asegura de que el proyecto sea técnicamente viable y rentable. En caso de que se crea que el proyecto no es viable, se detiene en esta fase.
    3. **Cimientos:** Durante la fase de Fundamentos, se establece una comprensión de alto nivel de la justificación del proyecto, la solución propuesta y la ejecución del proyecto. Se define el alcance del proyecto y se llega a un acuerdo sobre cómo, por quién, cuándo y dónde se llevará a cabo el proyecto. (debe durar pocas semanas y se puede volver a examinar durante el proyecto)

    En el caso de los proyectos pequeños, las fases de viabilidad y de cimientos pueden fusionarse en una sola.



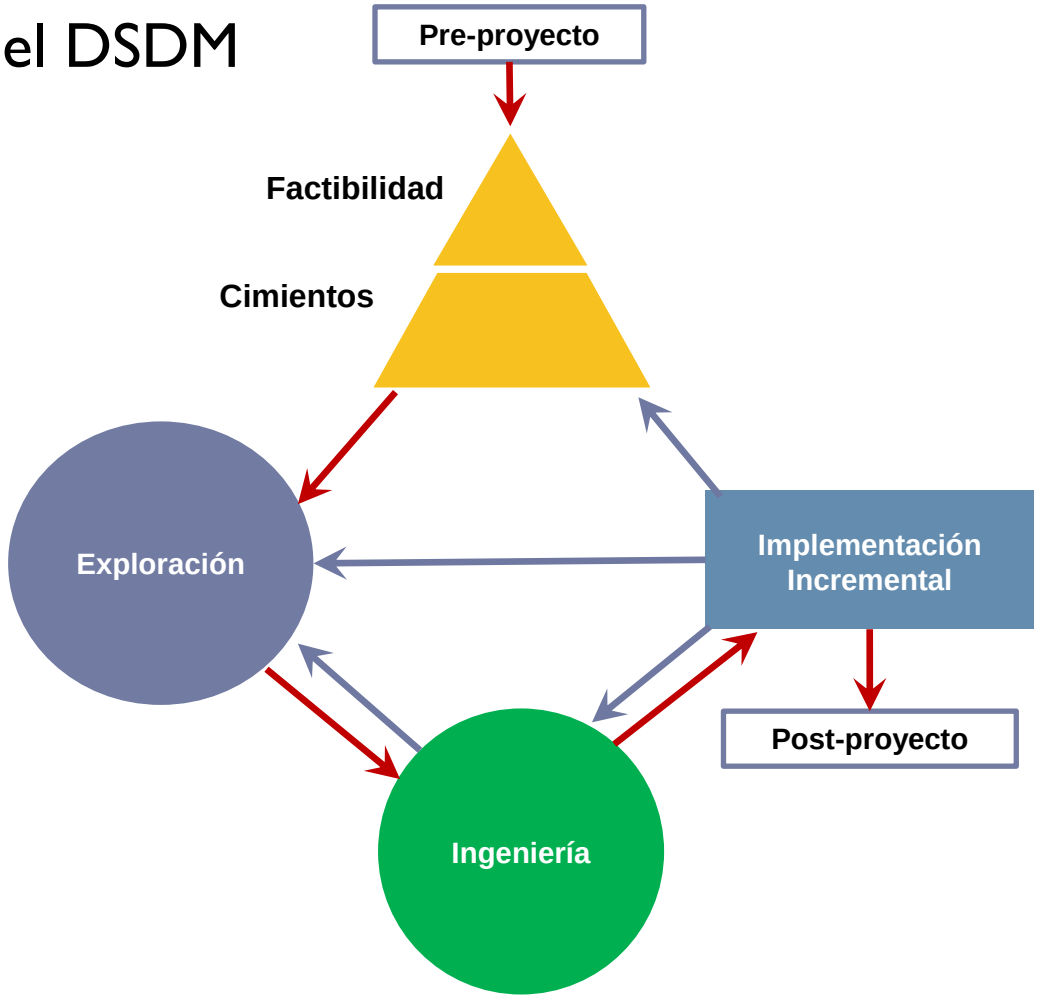
Modelos ágiles
Método de Desarrollo de Sistemas Dinámicos (DSDM)

- Enfoque y fases DSDM
  - Hay seis fases principales en el marco del DSDM:
    4. **Desarrollo evolutivo**: en esta fase la solución del proyecto es desarrollada en detalle por el Equipo de Desarrollo de Soluciones. El trabajo se lleva a cabo en cajas de tiempo.
    5. **Despliegue**: en esta fase la solución del proyecto se hace operativa. La solución puede desplegarse en su totalidad o en partes. La fase de Despliegue consiste en tres actividades principales: Ensamblar, revisar y desplegar. Después del último despliegue, el proyecto se cierra.
    6. **Post-proyecto**: en esta fase la organización evalúa si los beneficios esperados del proyecto han sido entregados. Para ello se crea un documento de evaluación de beneficios.



Modelos ágiles
Método de Desarrollo de Sistemas Dinámicos (DSDM)

- Fases del DSDM



Modelos ágiles
Método de Desarrollo de Sistemas Dinámicos (DSDM)

- **Ventajas:**
  - La calidad del producto es mojorada a través de la participación del usuario y de la naturaleza iterativa.
  - DSDM asegura un desarrollo ágil.
  - Disminuye el tiempo y el costo de los proyectos.
  - Permite realizar cambios fácilmente.
  - Permite la reutilización de las aplicaciones a través de los módulos existentes.



Modelos ágiles
Método de Desarrollo de Sistemas Dinámicos (DSDM)

- **Desventajas**:
  - Ningún sistema es realizado a la perfección en el primer intento.
  - Se necesita una alta participación de los usuarios para que los desarrolladores no asuman criterios que no son ciertos.
  - La entrega del producto deberá ser a tiempo, respetando presupuesto y asegurando calidad.
  - DSDM, requiere que se complete la iteración con la funcionalidad suficiente como para que inicie la siguiente iteración.
  - No es una metodología común y es algo difícil de entender.



Modelos ágiles
SCRUM

- **SCRUM**. Desarrollada por Ken Schwaber, Jeff Sutherland y Mike Beedle. Define un marco para la gestión de proyectos, que se ha utilizado con éxito durante los últimos 10 años.
- Está especialmente indicada para proyectos con un rápido cambio de requisitos. Sus principales características se pueden resumir en dos. El desarrollo de software se realiza mediante **iteraciones**, denominadas *sprints,* con una duración de 30 días. El resultado de cada sprint es un incremento ejecutable que se muestra al cliente. La segunda característica importante son las **reuniones a lo largo proyecto**, entre ellas destaca la reunión diaria de 15 minutos del equipo de desarrollo para coordinación e integración.



Modelos ágiles
SCRUM

- **Principios de Scrum para el éxito de los proyectos**
  - **Transparencia.** Los equipos trabajan en un entorno en el que todos son conscientes de los retos que pueden experimentar los demás. Las conversaciones periódicas cara a cara entre los miembros del equipo interdisciplinario y los propietarios del proyecto evitan la falta de comunicación y los cuellos de botella de la información.
  - **Reflexión.** Se incorporan puntos de reflexión frecuentes en el marco para permitir a los miembros del equipo revisar los avances logrados. Los gerentes de proyecto utilizan la información adquirida en estas reuniones de revisión para las estimaciones y la planificación de cara al futuro. Como resultado, los proyectos se pueden llevar a cabo de forma más eficiente, dentro del presupuesto y en el plazo previsto.
  - **Adaptación.** Los miembros del equipo pueden volver a priorizar las tareas en función de los requisitos cambiantes de los clientes. Deciden qué tareas se completan primero y cuáles se retoman en el futuro.



Modelos ágiles
SCRUM

- **Los equipos de trabajo de Scrum siguen cinco valores fundamentales.**
  - **Compromiso.** Los miembros del equipo de Scrum están comprometidos con las tareas y los objetivos basados en el tiempo y se dedican a la mejora continua para encontrar la mejor solución.
  - **Valentía.** Los equipos Scrum demuestran valentía al formular preguntas abiertas y que suponen un reto. Mantienen conversaciones honestas y transparentes para encontrar la mejor solución.
  - **Concentración.** Durante un periodo determinado, los miembros del equipo trabajarán a partir de las tareas del Product Backlog. Se centrarán en las tareas seleccionadas para proporcionar resultados en un plazo limitado.
  - **Actitud receptiva.** Los miembros del equipo de Scrum se muestran receptivos a las nuevas ideas y oportunidades que apoyan el aprendizaje individual y la calidad general del proyecto.
  - **Respeto.** Los miembros del equipo de trabajo respetan a los directores del proyecto, a los demás y al proceso Scrum. Esta cultura basada en el respeto crea un espíritu de colaboración y cooperación mutua dentro del equipo de trabajo.



Modelos ágiles
SCRUM

- La esencia de *Scrum* es un equipo organizado por cuenta propia que aporta valor al cliente en un período limitado conocido como *Sprint*. Scrum define los **artefactos**, los **roles** y los **eventos** asociados a cada Sprint.



Modelos ágiles
SCRUM

¿Qué son los artefactos de Scrum?

- Los artefactos de Scrum proporcionan información crítica de planificación y sobre la tarea tanto a los miembros del equipo de trabajo como a las partes interesadas.
  - **Product Backlog.** Se refiere a una lista dinámica de características, requisitos, mejoras y arreglos que se deben completar para que el proyecto tenga éxito. Consiste básicamente en la lista de quehaceres del equipo, que se revisa constantemente para ajustar las prioridades y adaptarse a los cambios que se producen en el mercado. El propietario del producto es quien mantiene y actualiza la lista. Está encargado de eliminar los elementos irrelevantes o de agregar nuevas solicitudes realizadas por los clientes.
  - **Sprint Backlog.** Se refiere a una lista de los elementos que el equipo de desarrollo debe completar durante el ciclo Sprint en curso. Antes de cada Sprint, el equipo escoge en qué elementos trabajará a partir del Product Backlog. Un Sprint Backlog es flexible y puede evolucionar durante un Sprint.
  - **Incremento.** Se refiere a un avance hacia la consecución de un logro o una visión. Es el producto final y utilizable que surge a partir de un Sprint. Los equipos pueden adoptar diferentes métodos para definir y demostrar los objetivos del Sprint. A pesar de la flexibilidad, el objetivo fundamental del Sprint, lo que el equipo desea alcanzar a partir del Sprint en curso, no puede verse comprometido.



Modelos ágiles
SCRUM

- Un **equipo de trabajo de Scrum** necesita tres roles específicos: un propietario de producto, un líder de Scrum y un equipo de desarrollo.
  - **Propietario de producto**
  - El propietario de producto se concentra en garantizar que el equipo de desarrollo proporcione el mayor valor al negocio. Comprende y prioriza las necesidades cambiantes de los usuarios finales y de los clientes. Los propietarios de producto exitosos trabajan así:
    - Suministran al equipo de trabajo orientación clara sobre qué características se deben entregar a continuación.
    - Tienden un puente entre lo que desea el negocio y lo que el equipo de trabajo entiende.
    - Deciden en qué momento y con qué frecuencia realizar los lanzamientos.



Modelos ágiles

SCRUM

- **Equipo de trabajo de Scrum**
  - **Líder de Scrum**
  - Los líderes de Scrum se encargan de velar por la metodología Scrum dentro de los equipos de trabajo. Son responsables por la efectividad del equipo de Scrum. Forman a los equipos, a los propietarios de los productos y al negocio para mejorar los procesos de Scrum y optimizar la entrega. Los líderes de Scrum también tienen las siguientes responsabilidades:
    - Programar los recursos necesarios para cada *Sprint*.
    - Organizar otros eventos del Sprint y las reuniones del equipo de trabajo.
    - Liderar la transformación digital dentro del equipo de trabajo.
    - Organizar las formaciones dirigidas al equipo al adoptar nuevas tecnologías.
    - Comunicarse con grupos externos para resolver cualquier reto al que se enfrente el equipo en su conjunto.



Modelos ágiles

SCRUM

- **Equipo de trabajo de Scrum**
  - **Equipo de desarrollo de Scrum**
  - El equipo de Scrum está formado por personas que realizan pruebas, diseñadores, especialistas en la experiencia del usuario, ingenieros de operaciones y desarrolladores. Los miembros del equipo cuentan con habilidades diferentes y se entrenan entre sí de modo que ninguna persona se convierta en un cuello de botella para la entrega del trabajo.
  - Los equipos de desarrollo de Scrum se ocupan de lo siguiente:
    - Trabajar de forma colaborativa para garantizar que el Sprint se complete con éxito.
    - Promover prácticas de desarrollo sostenible.
    - Organizarse por cuenta propia y abordar los proyectos con una actitud fraternal y solidaria evidente.
    - Controlar la planificación y estimar qué tanto trabajo se requiere para completar con éxito cada *Sprint*.



Modelos ágiles

SCRUM

- Los eventos de Scrum
- Los eventos o las ceremonias de Scrum son un conjunto de reuniones secuenciales que los equipos de Scrum sostienen periódicamente. Estos son algunos de los eventos de Scrum:
  - **Planificación del Sprint**
  - Durante este evento, el equipo estima el trabajo que se debe completar durante el siguiente Sprint. Los miembros definen los objetivos del Sprint, que a su vez deben ser específicos, medibles y viables. Tras la reunión de planificación, cada miembro del equipo de Scrum debe comprender cómo se puede entregar cada incremento durante el Sprint.
  - **Sprint**
  - Sprint se refiere al periodo durante el cual el equipo de Scrum trabaja conjuntamente para la consecución de un incremento. Habitualmente un Sprint dura dos semanas. Sin embargo, puede variar en función de las necesidades del proyecto y del equipo. Entre más complejo sea el trabajo y más factores desconocidos haya, el periodo del Sprint debe ser más corto.



Modelos ágiles

SCRUM

- Los eventos de Scrum
  - **Reunión de pie o diaria de Scrum**
  - La reunión diaria de Scrum es una sesión breve en la que los miembros del equipo se reportan y planifican el día. Informan sobre el trabajo concluido y expresan cualquier desafío para alcanzar los objetivos del Sprint. Se conoce como reunión de pie porque se busca que esta sea lo más breve posible, como cuando todos están de pie.
  - **Revisión del Sprint**
  - Al finalizar el Sprint, el equipo se reúne en el marco de una sesión informal para revisar el trabajo concluido y exponerlo ante las partes interesadas. Es posible que el propietario de producto también ajuste el Product Backlog en función del Sprint en curso.
  - **Retrospectiva del Sprint**
  - El equipo de trabajo se reúne para documentar y hablar sobre qué funcionó y qué no durante el Sprint. Las ideas generadas se utilizan para mejorar los próximos Sprints.



Modelos formales

- "Métodos Formales" se refiere a las técnicas y a las herramientas "matemáticamente rigurosas" para la especificación, el diseño y la Verificación de sistemas software y hardware "Matemáticamente rigurosas" significa que la especificación se realiza mediante enunciados bien formados en una lógica matemática y que la Verificación formal son deducciones rigurosas en esa lógica. La razón primordial para desarrollar técnicas de métodos formales es construir un marco dentro del cual se pueda predecir, de una manera científica, el comportamiento de los sistemas controlados por computadora.



Modelos formales

- Los métodos formales son una notación de especificación con una semántica formal, que se utiliza para especificar, analizar, diseñar, y verificar el software.
- Este enfoque requiere conocimientos matemáticos. Sin los mismos se pueden aplicar métodos formales, pero unos mínimos conocimientos facilitarán su uso y mejorarán el resultado final.



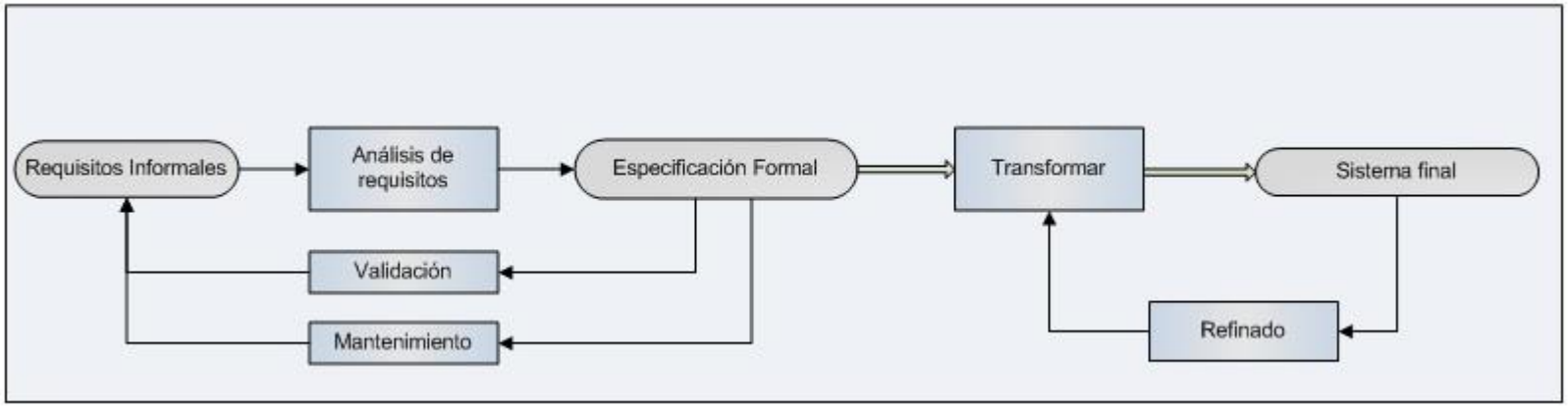
Modelos formales

- Fases:
  - Análisis de requisitos
  - Especificación formal
  - Transformación
  - Integración del sistema final
- La especificación formal se convierte en forma sistemática en una representación más detallada del sistema, matemáticamente correcta. Cada paso agrega detalle hasta que la especificación formal se convierte en un programa equivalente. Como hay muchos caminos a seguir desde la especificación hasta el sistema final, la secuencia de transformaciones y su justificación se reflejan en un registro formal de desarrollo. Se utilizan técnicas de validación del modelo matemático, como la Simulación.



Modelos formales

- La especificación de requisitos se refina en una especificación formal detallada, expresada en notación matemática. Los procesos de diseño, implementación y prueba de unidades se reemplaza por un proceso de transformaciones donde la especificación formal se refina hasta llegar a un Software.



Modelos formales

- Ventajas
  - Se comprende mejor el sistema.
  - La comunicación con el cliente mejora ya que se dispone de una descripción clara y no ambigua de los requisitos del usuario.
  - El sistema se describe de manera más precisa.
  - El sistema se asegura matemáticamente que es correcto según las especificaciones.
  - Mayor calidad software respecto al cumplimiento de las especificaciones.
  - Mayor productividad



Modelos formales

- Desventajas
  - El desarrollo de herramientas que apoyen la aplicación de métodos formales es complicado y los programas resultantes son incómodos para los usuarios.
  - Los investigadores por lo general no conocen la realidad industrial.
  - Es escasa la colaboración entre la industria y el mundo académico, que en ocasiones se muestra demasiado dogmático.
  - Se considera que la aplicación de métodos formales encarece los productos y ralentiza su desarrollo.



Modelos formales

- El uso de métodos formales está creciendo en el área del desarrollo de sistemas críticos, en donde las propiedades emergentes del sistema tales como seguridad, fiabilidad y protección son muy importantes. El alto costo de los fallos de funcionamiento en estos sistemas implica que las compañías están dispuestas a aceptar los costos elevados iniciales de los métodos formales para asegurar que su software es tan confiable como sea posible.

- Los sistemas críticos en los que los métodos formales se han aplicado con éxito incluyen un sistema de información de control de tráfico aéreo, sistemas de señalización de redes ferroviarias, sistemas de naves espaciales y sistemas de control médico.



Modelos formales

- La clasificación más común se realiza en base al modelo matemático subyacente en cada método, de esta manera podrían clasificarse en:

  - ***Especificaciones basadas en lógica de primer orden y teoría de conjuntos***: permiten especificar el sistema mediante un concepto formal de estados y operaciones sobre estados. Los datos y relaciones/funciones se describen en detalle y sus propiedades se expresan en lógica de primer orden. La semántica de los lenguajes está basada en la teoría de conjuntos.



Modelos formales

- ***Especificaciones algebraicas***: proponen una descripción de estructuras de datos estableciendo tipos y operaciones sobre esos tipos. Para cada tipo se define un conjunto de valores y operaciones sobre dichos valores. Las operaciones de un tipo se definen a través de un conjunto de axiomas o ecuaciones que especifican las restricciones que deben satisfacer las operaciones.



Modelos formales

- ***Especificación de comportamiento:***
  - ***Métodos basados en álgebra de procesos***: modelan la interacción entre procesos concurrentes. Esto ha potenciado su difusión en la especificación de sistemas de comunicación (protocolos y servicios de telecomunicaciones) y de sistemas distribuidos y concurrentes.
  - ***Métodos basados en Redes de Petri***: una red de petri es un formalismo basado en autómatas, es decir, un modelo formal basado en flujos de información. Permiten expresar eventos concurrentes. Los formalismos basados en redes de petri establecen la noción de estado de un sistema mediante lugares que pueden contener marcas. Un conjunto de transiciones (con pre y post condiciones) describe la evolución del sistema entendida como la producción y consumo de marcas en varios puntos de la red.



Modelos formales

- ***Especificación de comportamiento:***
  - ***Métodos basados en lógica temporal***: se usan para especificar sistemas concurrentes y reactivos. Los sistemas reactivos son aquellos que mantienen una continua interacción con su entorno respondiendo a los estímulos externos y produciendo salidas en respuestas a los mismos, por lo tanto el orden de los eventos en el sistema no es predecible y su ejecución no tiene por qué terminar.



Otros Modelos

Modelos estandarizados

- CMM (Capability Maturity Model)
- El modelo "clásico" en el tratamiento de la capacidad de los procesos de desarrollo y su madurez es CMM (Capability Maturity Model) del SEI (Software Engineering Institute). CMM tiene como objetivo evaluar los procesos en sus distintos niveles de madurez, identificar los niveles a través de los cuales una organización debe formarse para establecer una cultura de excelencia en la ingeniería de software.
- El modelo de madurez de procesos fue generado a través de la experiencia colectiva de los proyectos más exitosos de software, generando así un conjunto de prácticas importantes que deben ser implantadas por cualquier entidad que desarrolla o mantiene software.



Modelos estandarizados

- CMM es un marco de trabajo especificando guías para organizaciones de software que quieren incrementar su capacidad de procesos, considerando los siguientes puntos:
  - Identificar fortalezas y debilidades en la organización.
  - Identificar los riesgos de seleccionar entre diferentes contratos y monitorear los mismos.
  - Entender las actividades necesarias para planear e implementar los procesos de software.
  - Ayudar a definir e implementar procesos de software en la organización a través de una guía.
- Los procesos son evaluados a través de distintos niveles de madurez, que van desde prácticas desordenas o ad-hoc, hasta lograr una mejora continua de procesos y por ende de producto.



Modelos estandarizados

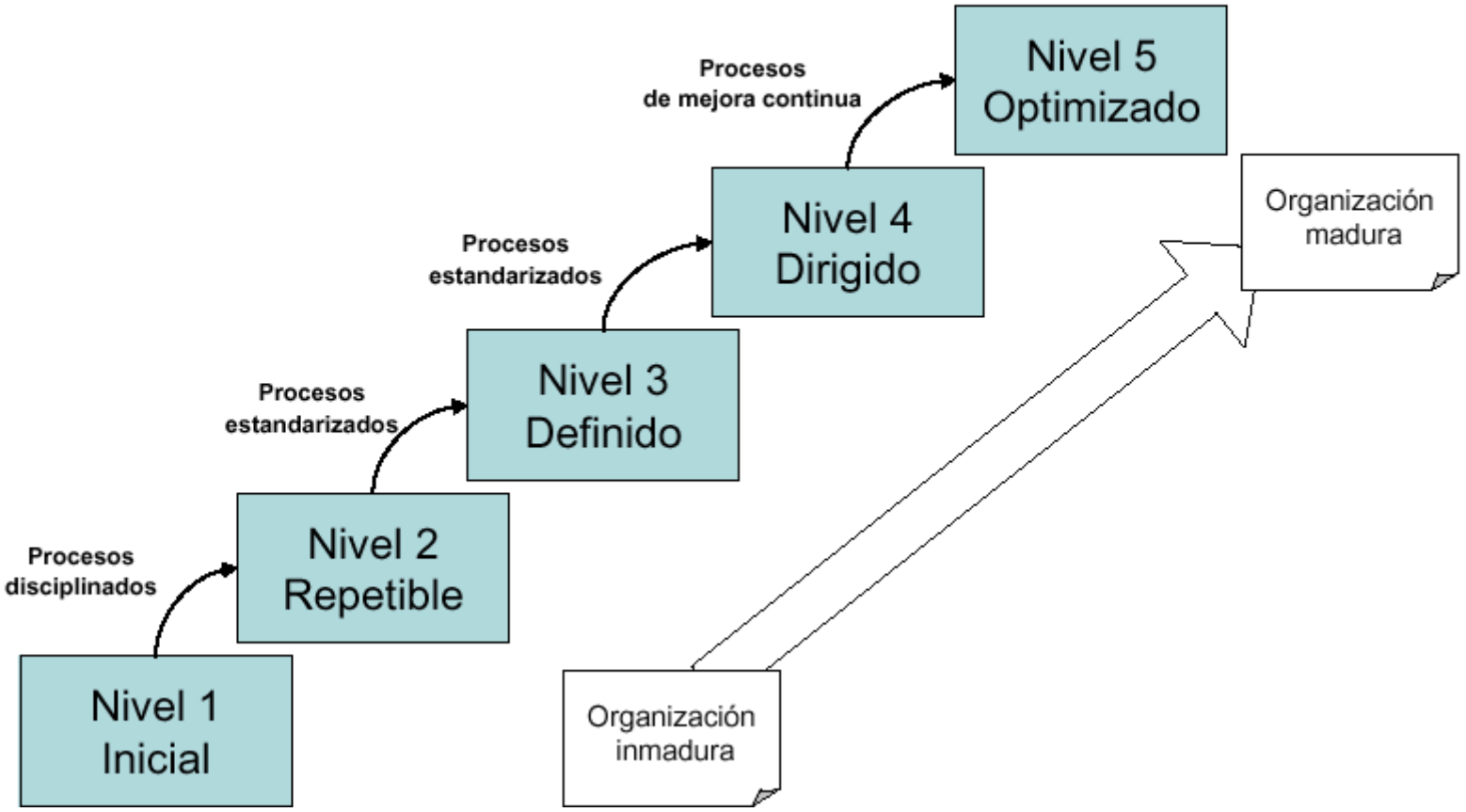
	Nivel	Características	Transición al siguiente nivel
1	Inicial.	Ad Hoc, poca formalización, junto con herramientas informalmente aplicadas al proceso.	Iniciar una administración rigurosa del proyecto, y asegurar la calidad
2	Repetible	Se logró un proceso estable con un nivel repetible de control estadístico.	Establecer un grupo de proceso y una arquitectura de proceso de desarrollo de software. Introducir métodos y tecnologías de ingeniería de software.
3	Definido	Se logró una base para un progreso mayor y continuo.	Establecer un conjunto básico de administraciones de proceso para identificar la calidad y costo de los parámetros y una base de datos del proceso. Juntar y mantener datos del proceso. Calcular la calidad relativa de cada producto e informar a la administración.
4	Administrado	Mejoras sustanciales en calidad junto con medidas comprensivas del proceso.	Apoyar la recopilación automática de datos del proceso. Usar datos para analizar y modificar el proceso.
5	Optimizado	Mejorías en base a mayor calidad y cantidad.	Continuar mejorando y optimizando el proceso.

Los 5 niveles del modelo de CMM.



Modelos estandarizados

El Modelo de Madurez de la Capacidad del Sw.



Modelos estandarizados

- ISO-9000
- El ISO-9000 (International Standard Organization) En contraste a CMM, la certificación ISO-9000 para la calidad del software fue adaptada de estándares generales y no incluye múltiples niveles, por lo cual o se está certificado o no se está. La certificación es equivalente al nivel 3 de la escala de SEI (Software Engineering Institute). Se calcula que hasta el año 1994, aproximadamente el 90% de las organizaciones de USA estaban por debajo del nivel 3.



Modelos estandarizados

- ISO-9000
- La madurez de los procesos es determinada por las distintas KPA (Key Process Area), es decir por las áreas básicas que componen a las organizaciones y su evolución. Al definir los procesos se tiene luego la capacidad de repetirlos, estandarizarlos en toda la organización, predecirlos para luego administrarlos (planificarlos, organizarlos, dirigirlos y controlarlos) y por último optimizarlos continuamente (hacerlos eficientes y eficaces).
- A pesar de lo anterior, la implementación de los modelos no ha sido cosa fácil, por ejemplo CMM a pesar de tener casi diez años de liberado sólo cuenta con 60 empresas en todo el mundo han logrado estar en el nivel 5 optimizado a Octubre del 2001 [Software Engineering Community, 2001].



Modelos estandarizados

- PSP/TSP
- El PSP (Personal Software Process) es una tecnología que tiene como justificación la premisa de que la calidad de software depende del trabajo de cada uno de los ingenieros de software y de aquí que el proceso diseñado debe ayudar a controlar, manejar y mejorar el trabajo de los ingenieros [Humphrey, 1998].
- El objetivo de PSP es lograr una mejor planeación del trabajo, conocer con precisión el desempeño, medir la calidad de productos y mejorar las técnicas para su desarrollo. La instrumentación de esta tecnología consiste en lo que se denomina "evolución del PSP". Se siguen ciertos pasos comenzando con las líneas base PSP0 y PSP0.1, el proceso personal de planeación PSP1 y PSP1.1, el manejo personal de calidad PSP2 y PSP2.1 y por último el proceso personal cíclico PSP3.



Modelos estandarizados

- **PSP0** (i) define el proceso de trabajo personal identificando y ordenando las principales (ii) introduce la recolección de datos para medir la productividad y calidad a través del registro de tiempo y defectos (iii) establece las bases para las mejoras en planificación de trabajo por tiempos y evaluación de resultados y (iv) documenta el proceso usando formas específicas. PSP0.1 (i) registra el tamaño del producto a través de puntos funcionales y estandarización de la codificación y (ii) registra los problemas y propuestas de mejora.
- **PSP1** (i) mejora la planeación introduciendo la estimación del tamaño del producto y (ii) introduce los reportes de pruebas. PSP1.1 (i) introduce las estimaciones de recursos e (ii) introduce la calendarización.
- **PSP2** (i) introduce las actividades de detección temprana de defectos a través de revisiones de diseño, código y uso de listas de verificación. PSP2.1 (i) introduce formas para el diseño detallado facilitando así la revisión del diseño.



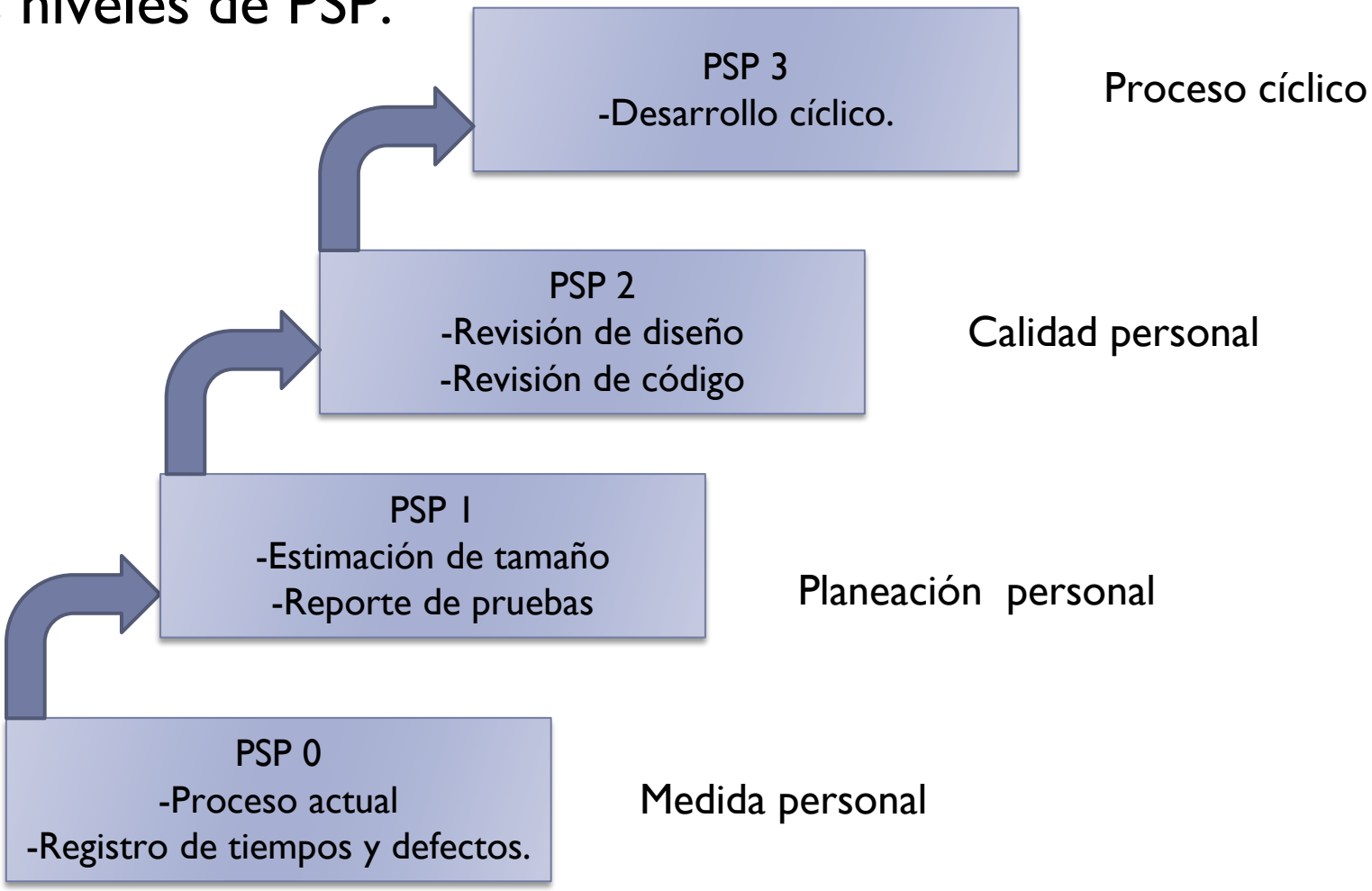
Modelos estandarizados

- **PSP3** (i) introduce el proceso cíclico para desarrollar programas de mayor tamaño, (ii) introduce el registro de seguimiento de asuntos y (iii) lleva el resumen de planeación y registro de tiempo, tamaño y defectos por ciclo.
- El PSP se considera la solución para pasar rápidamente entre niveles de CMM al lograr un mejor entendimiento de nuestras capacidades y habilidades y un mejor control sobre nuestro trabajo. Sin embargo, PSP tiene el problema de que es implementada a nivel individual. Al momento de la integración colectiva existen conflictos en el nivel organizativo, por lo cual se definió TSP (Team Software Process).
- El TSP se concentra en los aspectos del desarrollo de software realizados por equipos de trabajo, definiendo aspectos como la asignación y control de tareas para los diversos miembros del equipo.



Modelos estandarizados

- Los niveles de PSP.



Modelos estandarizados

- SPICE (Software Process Improvement and Capability dEtermination) [Dorling, 1995] es un modelo de madurez de procesos internacional.
- SPICE fomenta productos de calidad, promueve la optimización de procesos y facilita la evaluación del producto a través de los procesos de desarrollo. SPICE tiene diversos alcances, se aplica tanto a nivel directivo como a nivel de usuarios para asegurar que el proceso se encuentra alineado con las necesidades del negocio, apoya en que los proveedores de software tengan que someterse a una sola evaluación para aspirar a nuevos negocios y busca que las organizaciones de software dispongan de una herramienta universalmente reconocida para dar soporte a su programa de mejoramiento continuo.

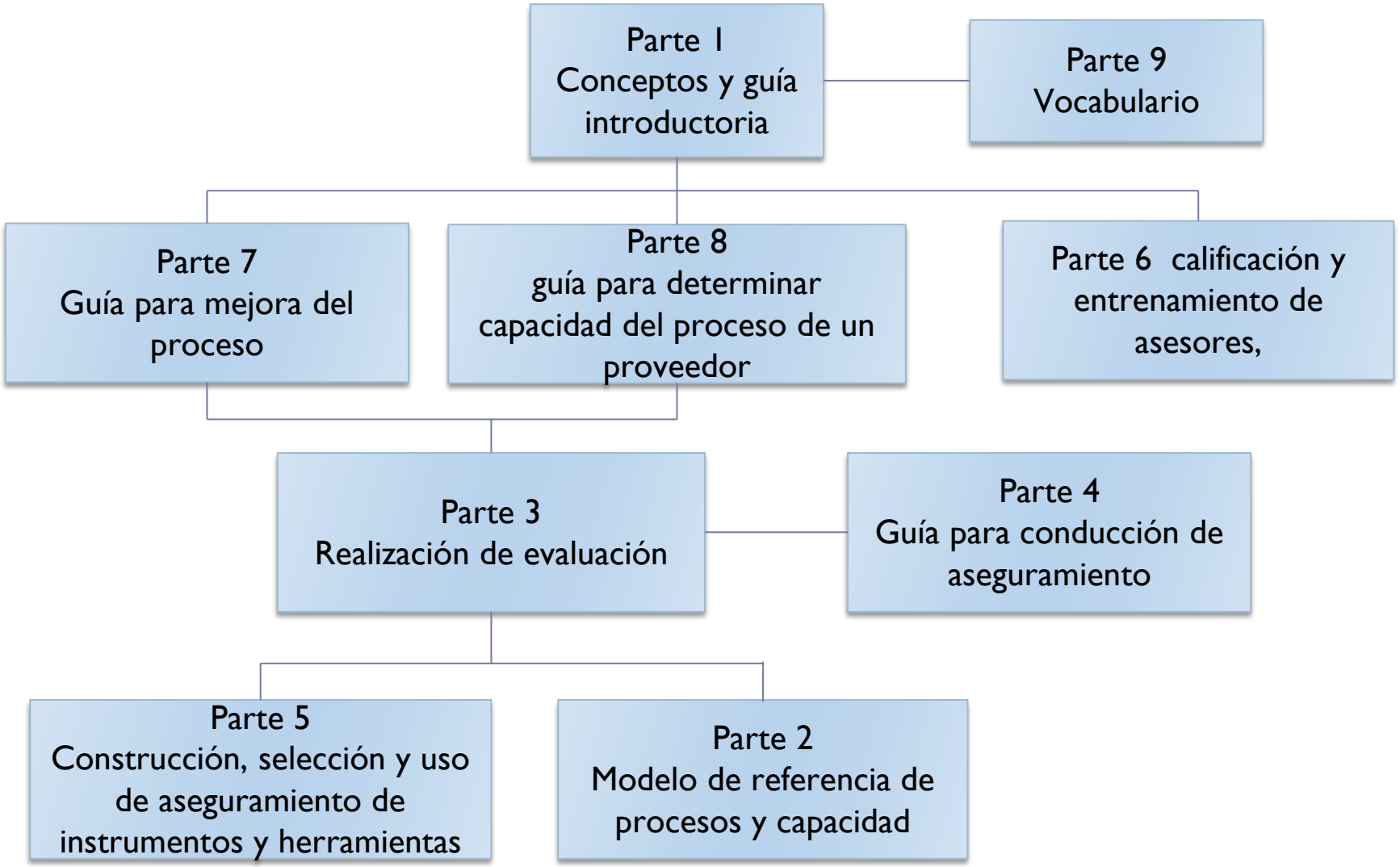


Modelos estandarizados

- SPICE tiene tres características principales: el marco de valor que contempla una dimensión funcional del procesos, la evidencia para la evaluación y la recurrencia dada por la selección de instancias de proyectos o productos.
- SPICE está conformado por 9 documentos que permiten instrumentar paso a paso el modelo con su correspondiente evaluación.
  - **Información del modelo.** (parte 1) conceptos y guía introductoria, (parte 4) guía para conducción de aseguramiento, (parte 6) calificación y entrenamiento de asesores, (parte 7) guía para mejora del proceso, (parte 8) guía para determinar capacidad del proceso de un proveedor y (parte 9) vocabulario general.
  - **Normatividad del modelo.** (parte 2) modelo de referencia de procesos y capacidad, (parte 3) realización de evaluación, (parte 5) construcción, selección y uso de aseguramiento de instrumentos y herramientas.



Modelos estandarizados



- Componentes del modelo SPICE

Modelos estandarizados

- Al igual que CMM (CMMi - Capability Maturity Model Integrated), SPICE integra una serie de niveles por la que sus procesos deberán pasar para obtener cómo resultado final la madurez. Los niveles son: Nivel 0 Incompleto, Nivel 1 Fabricado informalmente, Nivel 2 Planeado, Nivel 3 Bien definido, Nivel 4 Controlado cuantitativamente, y Nivel 5 Mejora continua.
- Adicionalmente hay una definición de procesos generales que abarcan a toda la organización y a través de los cuáles se identifica el cómo lograrlos: Cliente – Proveedor CUS, Ingeniería ENG, Administración MAN, Apoyo o soporte SUP y Organización ORG. Los procesos generales son soportados por prácticas específicas que deberán cumplirse para lograr un paso de niveles, además de la estrecha relación entre los mismos.



Modelos estandarizados

- SPICE hace hincapié en la calidad y actualización, así como en la vigencia del producto. Ya que la tecnología es cambiante, las fases que marca el modelo SPICE son sin duda uno de los pilares en que se tendrá que trabajar con la mayor dedicación para obtener calidad en el producto y que el servicio del mismo sea excelente, además de generar la confianza necesaria hacia la dirección y hacia el usuario de donde se obtiene la información.



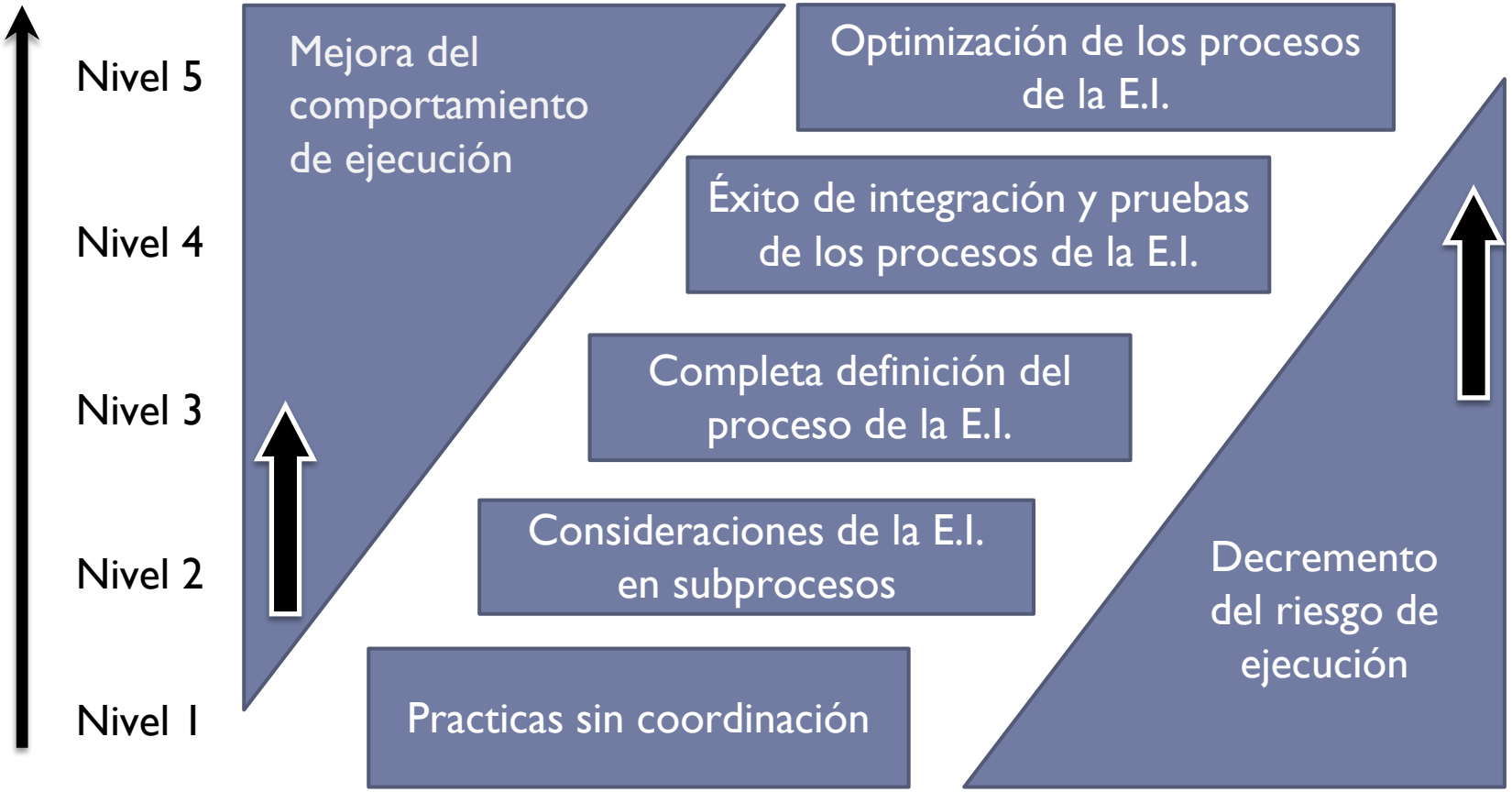
Modelos estandarizados

- PEMM (Performance Engineering Maturity Model) [Scholz, 1999] presenta un modelo para evaluar los niveles de integración, aplicación, ejecución y diseño, llamado ingeniería de la ejecución del modelo de madurez. Al igual que SPICE se apoya en el modelo de madurez de capacidades CMM. El objetivo de PEMM es poder evaluar la Ejecución de la Ingeniería (EI) así como la integración del proceso. El modelo sirve tanto para evaluar una organización como los propios desarrollos de procesos tecnológicos específicos. Sirve también para definir el criterio al escoger un proveedor de software para los productos críticos o semi-críticos de la compañía.
- Al igual que el CMM, PENN cuenta con 5 niveles, los cuales determinan la mejora del comportamiento de ejecución y el decremento del riesgo de ejecución a través de estos niveles



Modelos estandarizados

- Modelo general PEMM



Modelos estandarizados

- La evaluación de una compañía se hace a través de la medición de aspectos generales, la organización, la definición de procesos de ingeniería, el proyecto de la dirección y la tecnología, a través de 34 preguntas, utilizando el método Meta-Pregunta-Métrico (GQM, Goal Question Metric). GQM se usa para encuestas expertas identificando y midiendo los objetivos a través de preguntas con respuestas cuantificables (en la actualidad sólo ' Si' o ' No').



Ganar - Ganar

- El modelo "Win-Win" [Boehm 1998] se basa en el modelo de espiral y da énfasis en la identificación de las condiciones de ganancia para todas las partes implicadas. Se crea un plan para alcanzar las condiciones ganadoras, determinando los riesgos involucrados.
- El principal objetivo del modelo es establecer las reglas para la definición del proceso de desarrollo del proyecto tomando en cuenta a todos los implicados.



Ganar - Ganar

- Ganar - Ganar
  - Son cuatro los ciclos del modelo consistiendo de cuatro actividades principales cada uno:
    - Definición de los objetivos del proceso y elaboración del sistema y subsistemas del producto.
    - Evaluación de las alternativas con respecto a los objetivos del proyecto. Identificación y resolución de las fuentes principales de riesgo en el proceso de desarrollo de los productos.
    - Elaboración de la definición de los productos y procesos.
    - Planeación del siguiente ciclo. Calendarización del ciclo de vida del plan, incluyendo la partición del sistema en subsistemas para llevar el proceso en ciclos paralelos.



Ganar - Ganar

- Ciclo 0. Grupo de aplicación: Se determina la viabilidad de un grupo apropiado de aplicaciones.
- Ciclo 1. Objetivos del ciclo de vida de la aplicación: Se desarrollan objetivos del ciclo de vida, incluyendo prototipos, planes y especificaciones de aplicaciones individuales, y se verifica la existencia de al menos una arquitectura viable para cada aplicación.
- Ciclo 2. Arquitectura del ciclo de vida de la aplicación: Se establece una arquitectura del ciclo de vida detallado, se verifica la viabilidad y determina que no existen riesgos mayores en satisfacer los planes y especificaciones.
- Ciclo 3. Capacidad de operación inicial: Alcanzar una capacidad operacional inicial para cada etapa crítica del proyecto en el ciclo de vida del software.



Ganar - Ganar

- Las máximas o creencias del modelo son las siguientes:
  1. Crear software basado en componentes para lograr mayor calidad en sistemas de mayor tamaño.
  2. Escribir software reutilizable para hacer eficiente el proceso de desarrollo.
  3. Medir la calidad del sistema como aspecto clave del desarrollo del producto.
  4. Lograr mayor calidad en el proceso de ensamblaje a partir de componentes menores.
  5. Usar tecnología basadas objetos como aspecto básico para lograr la calidad.
  6. Poder lograr sistemas más rápidamente, sencillos, confiables y de calidad a través de procesos bien definidos.



Ganar - Ganar

- Las máximas o creencias del modelo son las siguientes:
  7. Utilizar el modelo de espiral como base del proceso.
  8. Flexibilizar el proceso de desarrollo del software para lograr los objetivos generales de eficiencia.
  9. Involucrar al cliente mediante el manejo de prototipos.
  10. Analizar los riesgos en el proceso del desarrollo del software para asegurar la calidad final del sistema.
- No hay límite en el alcance o tipo de proyectos donde pueda ser aplicado el modelo "Win-Win". No se necesita mucho tiempo de gestión, de forma que se puede utilizar en proyectos pequeños, tanto como proyectos más grandes



Modelo basado en componentes reutilizables

- Ingeniería del software basada en componentes. Este enfoque se basa en la existencia de un número significativo de componentes reutilizables. El proceso de desarrollo del sistema se enfoca en integrar estos componentes en el sistema más que en desarrollarlos desde cero.



Modelo basado en componentes reutilizables

- Reutilización de software.
  - Código similares los buscan, los modifican según lo creen necesario y los incorporan en el sistema.
  - Tiene la ventaja obvia de reducir la cantidad de software a desarrollarse y así reduce los costos y los riesgos.
  - las etapas en el proceso de la reutilización :
    - Análisis de componentes
    - Modificación de requerimientos
    - Diseño del sistema con reutilización
    - Desarrollo e integración

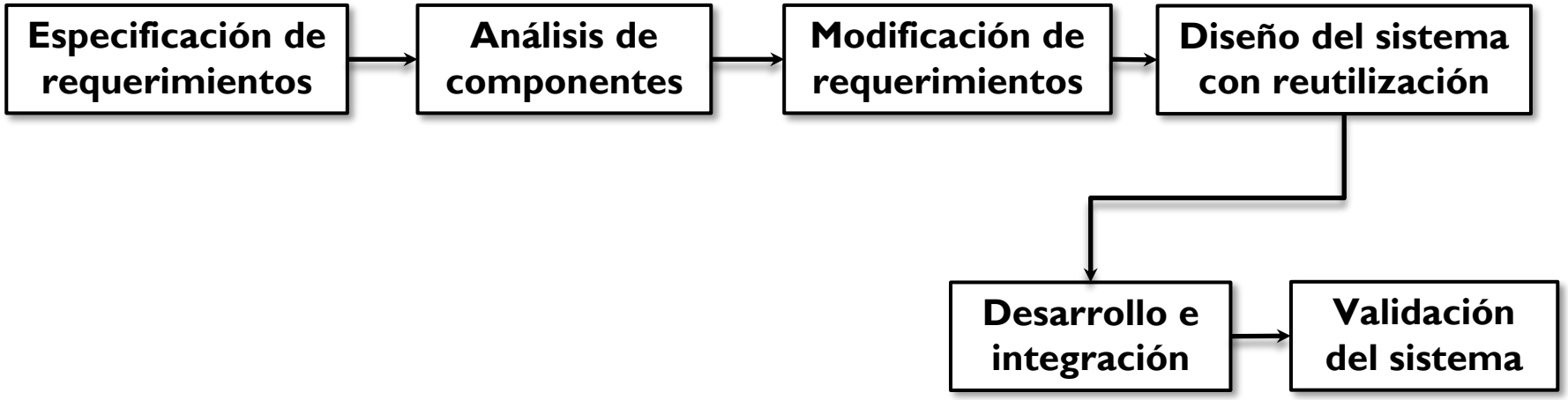


Modelo basado en componentes reutilizables

- **Análisis de componentes**
  - Según la especificación buscan componentes para implementar esta especificación.
- **Modificación de requerimientos**
  - Se analizan los requerimientos, los componentes encontrados se modifican.
- **Diseño del sistema con reutilización**
  - Se diseña o se reutiliza un marco de trabajo.
- **Desarrollo e integración**
  - El software que no se puede adquirir externamente se desarrolla.



Modelo basado en componentes reutilizables





Modelo basado en componentes reutilizables

- Es utilizado para reducir los costos, tiempo y esfuerzos de desarrollo del software, y de esta manera incrementar el nivel de productividad de los grupos desarrolladores y minimizar los riesgos; a su vez ayuda a optimizar la fiabilidad, flexibilidad y la reutilización de la aplicación final.



Modelo basado en componentes reutilizables

- La modularidad, la reusabilidad y componibilidad son características muy relevantes de la tecnología de programación basada en componentes, en las cuales coincide con la tecnología orientada a objetos de la que puede considerarse una evolución. No obstante en esta tecnología también se requiere robustez debido a que los componentes deben operar en entornos muchos más heterogéneos.



Modelo basado en componentes reutilizables

- El desarrollo de software basado en componentes (DSBC), se corresponde al paradigma de programación de sistemas abiertos, los cuales son extensibles y tienen una interacción con componentes heterogéneos que se integran o abandonan el sistema de manera dinámica, o sea, que los componentes pueden ser substituidos por otros componentes, independientemente de su arquitectura y desarrollo.



Modelo basado en componentes reutilizables

- Otro aspecto a tener en cuenta en el DSBC, es el poder integrar lo mejor de las tecnologías para realizar aplicaciones personalizadas, ajustadas a los requisitos de los clientes; lo cual le permite a los desarrolladores y/o a la empresa adquirir las tecnologías que más se adapten a sus particularidades, sin incurrir en gastos de licenciamiento o soporte y actualización de grandes soluciones.



Modelo basado en componentes reutilizables

- Ventajas:
  - **Simplifica las pruebas**. Permite que las pruebas sean ejecutadas probando cada uno de los componentes antes de probar el conjunto completo de componentes ensamblados.
  - **Simplifica el mantenimiento del sistema**. Cuando existe un débil acoplamiento entre sus componentes, el desarrollador puede actualizar y/o adicionar componentes según sea requerido, sin afectar otras partes del sistema.
  - **Mayor calidad**. Dado que un componente puede ser construido y luego optimizado continuamente por un experto u organización, la calidad de una aplicación basada en componentes mejorará con el paso del tiempo.
  - **Ciclos de desarrollo más cortos**. La adición de una pieza dada de funcionalidad tomará días en lugar de tardar meses ó incluso años.
  - **Mejor ROI**. Usando correctamente esta estrategia, el retorno sobre la inversión puede ser más favorable que desarrollando los componentes uno mismo.
  - **Funcionalidad mejorada**. Para usar un componente que contenga una pieza de funcionalidad, solo se necesita entender su naturaleza, más no sus detalles internos.



Conclusiones

- Una vez conocido el proceso a automatizar en sw, se debe elegir el modelo más apropiado de desarrollo de sw.
- El modelo de cascada considera cada actividad del proceso como una actividad discreta.
- El modelo de desarrollo evolutivo considera actividades del proceso en forma concurrente.
- El modelo de espiral se basa en análisis de riesgos.
- La visibilidad del proceso involucra la creación de documentos o resultados de las actividades.
- Los Ingenieros de software deben tener responsabilidades éticas, sociales y profesionales.



Bibliografía

- Pressman, S Roger (2005) “Ingeniería del Software: Un enfoque práctico”, 6a edición McGraw-Hill.
- Somerville, Ian (2005) “Ingeniería de software”. 7a edición. Addison Wesley.

